Automata and Formal Languages

Adam Kelly (ak2316@cam.ac.uk)

July 27, 2026

We all know an algorithm when we see one, and for most of mathematics that is enough. But some questions cannot be answered without saying exactly what an algorithm *is*: if we want to prove that no algorithm solves a certain problem, we have to quantify over all possible algorithms, and to do that we need a definition. This course provides one – in fact several, which turn out to agree – and then uses it to show that a great many natural problems have no algorithmic solution at all.

Running alongside this is the theory of *formal languages*. A language here is just a set of finite strings over a fixed alphabet, and this is a flexible enough notion to encode almost anything: sets of numbers, sets of graphs, sets of programs. Chomsky's insight was that languages come in a hierarchy, graded by how complicated a machine is needed to recognise them, and much of what follows is an exploration of the bottom two levels of that hierarchy – the regular languages, recognised by finite automata, and the context-free languages, recognised by automata with a stack – before we climb to the top and meet the languages recognised by a genuinely general-purpose computer.

This article constitutes my notes for the 'Automata and Formal Languages' course, held in Michaelmas 2022 at Cambridge. These notes are *not a transcription of the lectures*, and differ significantly in quite a few areas. Still, all lectured material should be covered.

Contents

§1 Introduction

§1.1 Algorithms and the Entscheidungsproblem

There is a difference between knowing that something exists and being able to find it. Compare the two statements

'every polynomial of degree $n \geq 1$ over $\mathbb{C}$ has a root'

and

'there is a procedure which, given such a polynomial, produces a root'.

The first is the fundamental theorem of algebra. The second is a genuinely different assertion, and there are plenty of situations in mathematics where the existence proof is available but no procedure is.

Historically, the demand for procedures came from Hilbert. In his 1900 address in Paris he set out a list of problems for the coming century, the tenth of which asks for a procedure which, given a Diophantine equation (a polynomial with integer coefficients), decides whether it has an integer solution. In 1928, in the book *Grundzüge der theoretischen Logik* of Hilbert and Ackermann, the **Entscheidungsproblem** ('decision problem') was posed: give a procedure which, given a formula of predicate logic, decides whether it is a tautology.

In both cases Hilbert expected a positive answer, and here is the crucial asymmetry. To answer such a question positively one does not need a definition of 'algorithm' at all: one simply writes an algorithm down, and everyone agrees on seeing it that it is one. To answer it negatively, one must rule out every conceivable algorithm, and for that a precise definition is unavoidable. Both of Hilbert's questions turned out to have negative answers – Church and Turing settled the Entscheidungsproblem in 1936, and Matiyasevich settled the tenth problem in 1970 – and neither answer could have been given before the work we do in this course.

§1.2 Alphabets, Words and Languages

What are the objects on which computation is performed? One's first guess is 'numbers', but neither of the two problems above has numbers as its input: one takes polynomials and the other takes formulae. What is common to them is that the inputs can be written down as finite strings of symbols drawn from a fixed finite stock. This is how all modern computation works, and it is the setting we adopt.

Definition 1.1 (Strings)

Let Ω be a set, whose elements we call **symbols**. For $n \in \mathbb{N}$ we write Ω^n for the set of functions $n \to \Omega$, thinking of the natural number n as the set $\{0, 1, \dots, n-1\}$ in the usual von Neumann way, so that an element of Ω^n is a sequence $(x_0, \dots, x_{n-1})$ of symbols. We write

$$\Omega^\star = \bigcup_{n \in \mathbb{N}} \Omega^n$$

for the set of all finite sequences of symbols, which we call **Ω-strings**.

We will suppress the sequence notation entirely and write strings by juxtaposition, so that (x_0, x_1, x_2) is written $x_0x_1x_2$. The following conventions will be used without

comment.

- The **length** of a string $\alpha \in \Omega^n$ is $|\alpha| = n$.
- Ω^0 has exactly one element, the **empty string** ε.
- Given $\alpha \in \Omega^n$ and $k \leq n$, the **truncation** $\alpha \restriction_k$ is the unique element of Ω^k which is an initial segment of α.
- Given $\alpha \in \Omega^m$ and $\beta \in \Omega^n$, their **concatenation** $\alpha\beta \in \Omega^{m+n}$ is defined in the obvious way. Concatenation is associative with identity ε, so $\Omega^\star$ is a monoid.
- We define $\alpha^0 = \varepsilon$ and $\alpha^{n+1} = \alpha\alpha^n$.
- We identify a symbol $x \in \Omega$ with the string of length one that it determines.
- If $Y, Z \subseteq \Omega^\star$ then $YZ = \{\alpha\beta \mid \alpha \in Y,\, \beta \in Z\}$, and if $Y = \{\alpha\}$ we simply write αZ.
- A function $f\colon \Omega \to \Omega'$ lifts to a function $\Omega^\star \to \Omega'^\star$ acting symbolwise; we write this lift as f as well.

Now, not every string is a sensible object. If Ω is a stock of letters then $\Omega^\star$ contains a great many strings which are not words of English; if Ω is a stock of words then $\Omega^\star$ contains a great many strings which are not sentences. So we will want to designate a subset of $\Omega^\star$ as the *well-formed* strings, and it is these subsets which are the objects of study.

Definition 1.2 (Alphabet, word, language)

An **alphabet** is a finite nonempty set Σ, whose elements we call **letters** and denote $a, b, c, \ldots$. We write

$$\mathbb{W} = \Sigma^\star$$

for the set of **words**, denoted $u, v, w, \ldots$, and $\mathbb{W}^+ = \mathbb{W} \setminus \{\varepsilon\}$ for the set of nonempty words. A **language** over Σ is a subset $L \subseteq \mathbb{W}$.

Because a language is simply a set of words, and words can encode anything, this is a very general notion indeed. The set of prime numbers written in binary is a language over $\{0, 1\}$; so is the set of (codes of) computer programs that never terminate. The whole of this course can be read as the study of how complicated a language can be.

§1.3 Counting Languages

We record here some counting facts from Numbers and Sets which will be used repeatedly, usually to demonstrate that some class of languages cannot possibly contain all of them. Recall that X is **countable** if there is a surjection $\mathbb{N} \to X$ (or $X = \varnothing$), and that X is **infinite** if there is an injection $\mathbb{N} \to X$.

Proposition 1.3

If X is nonempty and countable then $X^\star$ is countably infinite.

Proof. Pick $x \in X$. The map $n \mapsto x^n$ is an injection $\mathbb{N} \to X^\star$, so $X^\star$ is infinite.

For countability, let $\pi\colon \mathbb{N} \to X$ be a surjection. Every $k \in \mathbb{N}$ with $k > 0$ has a

unique prime factorisation $k = \prod_i p_i^{k_i}$, where $p_0 = 2, p_1 = 3, p_2 = 5, \ldots$ enumerates the primes. Read k_0 as a length and send k to the string

$$\pi(k_1)\,\pi(k_2)\cdots\pi(k_{k_0}) \in X^\star,$$

sending 0 to ε as well. Given any string $\alpha = y_1 \cdots y_n \in X^\star$, choose m_j with $\pi(m_j) = y_j$ and take $k = 2^n 3^{m_1} 5^{m_2} \cdots$; this maps onto α. So the map is a surjection $\mathbb{N} \to X^\star$. □

Proposition 1.4

If X is countable then the set $\mathrm{Fin}(X)$ of finite subsets of X is countable.

Proof. The 'forgetful' map $X^\star \to \mathrm{Fin}(X)$ sending $y_1 \cdots y_n$ to $\{y_1, \ldots, y_n\}$ is a surjection: given a finite $F \subseteq X$, list its elements in any order. Composing with the surjection of Proposition 1.3 gives a surjection $\mathbb{N} \to \mathrm{Fin}(X)$. □

On the other hand, by Cantor's theorem $\mathcal{P}(X)$ is uncountable whenever X is infinite. Since $\mathbb{W}$ is countably infinite, we obtain the following, which is the first thing worth knowing about languages.

Corollary 1.5

There are uncountably many languages over any alphabet.

Every method we shall meet for *specifying* a language – a grammar, an automaton, a regular expression, a register machine – is a finite object over some finite alphabet, and so there are only countably many of them. Consequently *every* such method fails to describe almost every language. This is not a defect of any particular formalism; it is a fact about cardinality, and it is worth keeping in mind throughout.

§2 Rewrite Systems and Grammars

§2.1 Rewrite Systems

The most primitive way of generating strings is to start with one and repeatedly replace pieces of it. This is the notion of a rewrite system, and everything in the first half of the course is a special case of it.

Definition 2.1 (Rewrite system)

Let Ω be a finite set of symbols. An element (α, β) of $\Omega^\star \times \Omega^\star$ is called a **rewrite rule** or **production rule**, and is written $\alpha \to \beta$. A pair $R = (\Omega, P)$ is a **rewrite system** if P is a *finite* set of rewrite rules.

The intended reading of $\alpha \to \beta$ is 'wherever you see α, you may replace it by β'. Let us make that precise.

Definition 2.2 (Derivation)

Let $R = (\Omega, P)$ be a rewrite system and let $\sigma, \tau \in \Omega^\star$. We write

$$\sigma \Rightarrow_R \tau$$

and say that R **produces** τ **from** σ **in one step**, if there are $\alpha, \beta, \gamma, \delta \in \Omega^\star$ with $\sigma = \alpha\gamma\beta$, $\tau = \alpha\delta\beta$ and $\gamma \to \delta \in P$. We write $\Rightarrow_R^\star$ for the reflexive transitive closure of $\Rightarrow_R$. A sequence

$$\sigma_0 \Rightarrow_R \sigma_1 \Rightarrow_R \cdots \Rightarrow_R \sigma_n$$

is an R**-derivation of** σ_n **from** σ_0 **of length** n, and we write

$$\mathcal{D}(R, \sigma) = \{\tau \in \Omega^\star \mid \sigma \Rightarrow_R^\star \tau\}$$

for the set of strings derivable from σ.

Note that α and β here are the *context* in which the rewriting takes place; the rule fires regardless of what surrounds the occurrence of γ. We will meet later a restricted notion in which the context is allowed to constrain the rule, and this is the origin of the word 'context-free'.

Since a rewrite system is a finite object over a finite alphabet, Propositions 1.3 and 1.4 give the following immediately.

Proposition 2.3

For fixed finite Ω, there are only countably many rewrite systems on Ω.

Proof. $\Omega^\star$ is countable, hence so is $\Omega^\star \times \Omega^\star$, hence so is $\mathrm{Fin}(\Omega^\star \times \Omega^\star)$, and every P lies in the latter. □

§2.2 Grammars

The application we care about is to languages. Chomsky's idea was that a natural language should be modelled not by a finite list of well-formed sentences, but by a finite list of *rules* which generate infinitely many sentences. The reason for insisting on infinitude is **linguistic recursion**: if X is a grammatical English sentence then so is 'Alice believes that X', and if we impose a maximum sentence length we are obliged to cut this off at some arbitrary point.

Here is a toy example of the sort of thing Chomsky had in mind. Take the rules

$$S \to \mathrm{NP\ VP}, \qquad \mathrm{NP} \to \mathrm{Adj\ NP}, \qquad \mathrm{NP} \to \mathrm{Noun},$$
$$\mathrm{VP} \to \mathrm{Verb}, \qquad \mathrm{VP} \to \mathrm{Verb\ Adv},$$

together with rules turning Adj into an adjective, Noun into a noun and so on. Starting from S one can derive 'colourless green ideas sleep furiously', Chomsky's famous example of a sentence that is grammatical but meaningless; one cannot derive 'furiously sleep ideas green colourless', which is neither. The moral is that grammaticality is a formal, checkable property, whereas meaningfulness is not something a rewrite system can see.

The symbols in this example fall into two classes: those like S and NP which are scaffolding, and those like 'ideas' which are part of the final product. This distinction is built into the definition.

Definition 2.4 (Grammar)

Let Σ be an alphabet and let V be a nonempty finite set disjoint from Σ, whose elements we call **variables** or **nonterminal symbols**, denoted $A, B, C, \ldots$. Write $\Omega = \Sigma \cup V$. A **grammar** is a tuple $G = (\Sigma, V, P, S)$ where (Ω, P) is a rewrite system and $S \in V$ is the **start symbol**.

We use the notation for rewrite systems for the associated rewrite system, writing $\Rightarrow_G$, $\Rightarrow_G^\star$ and $\mathcal{D}(G, \sigma)$. The **language generated by** G is

$$\mathcal{L}(G) = \mathcal{D}(G, S) \cap \mathbb{W}.$$

So a word lies in $\mathcal{L}(G)$ exactly when it can be derived from the start symbol and contains no variables. Two easy ways for this to fail are worth noting straight away.

Example 2.5

If P contains no rule of the form $S \to \alpha$, then $\mathcal{D}(G, S) = \{S\}$, which contains no words, so $\mathcal{L}(G) = \varnothing$. Similarly, if no rule has a right-hand side lying in $\mathbb{W}$, then nothing in $\mathcal{D}(G, S)$ can be a word, and again $\mathcal{L}(G) = \varnothing$.

Example 2.6

Let $\Sigma = \{a\}$, $V = \{S\}$ and $P_0 = \{S \to aaS,\ S \to a\}$. We claim that

$$\mathcal{L}(G_0) = \{a^{2n+1} \mid n \in \mathbb{N}\}.$$

Each rule replaces a string by one of length two greater, or of the same length, so by induction on the length of a derivation every element of $\mathcal{D}(G_0, S)$ has odd length; in particular every word produced is a^{2n+1} for some n. Conversely, applying $S \to aaS$ exactly n times and then $S \to a$ produces a^{2n+1}.

Observe that the only feature of P_0 used in the argument was that every rule preserves the parity of the length. So the rule sets

$$P_1 = \{S \to aSa,\ S \to a\}, \quad P_2 = \{S \to Saa,\ S \to a\}, \quad P_3 = \{S \to aaS,\ S \to aaSaa,\ S \to a\}$$

all generate the same language. Different grammars can generate the same language, and this is the phenomenon we look at next.

§2.3 Equivalence of Grammars

Definition 2.7 (Equivalent grammars)

Grammars G and G' over the same alphabet are **equivalent** if $\mathcal{L}(G) = \mathcal{L}(G')$.

Our aim in this subsection is to show that, for a fixed alphabet Σ, there are only countably many languages of the form $\mathcal{L}(G)$. The point requiring care is that V is allowed to be an arbitrary finite set, so there is a proper class of grammars to begin with; we must cut this down to a set. The tool is the observation that the names of the variables do not matter.

Definition 2.8 (Isomorphism of grammars)

Let $G = (\Sigma, V, P, S)$ and $G' = (\Sigma, V', P', S')$ be grammars over the same alphabet. A function $f : \Omega \to \Omega'$ is an **isomorphism** if

(i) $f \restriction_\Sigma = \mathrm{id}$;

(ii) $f(S) = S'$;

(iii) $f \restriction_V$ is a bijection $V \to V'$;

(iv) $\alpha \to \beta \in P$ if and only if $f(\alpha) \to f(\beta) \in P'$,

where in (iv) we use the lift of f to $\Omega^\star$.

Proposition 2.9

Isomorphic grammars are equivalent.

Proof. If f is an isomorphism $G \to G'$ then f^{-1} is an isomorphism $G' \to G$, so it suffices to prove $\mathcal{L}(G) \subseteq \mathcal{L}(G')$. Let $w \in \mathcal{L}(G)$, with derivation

$$S = \sigma_0 \Rightarrow_G \sigma_1 \Rightarrow_G \cdots \Rightarrow_G \sigma_n = w.$$

Apply f throughout. By (ii) the first term is S'; by (i) the last term is still w, since w contains only letters; and by (iv) each single step remains a legal step, since the decomposition $\sigma_i = \alpha\gamma\beta$ maps to $f(\sigma_i) = f(\alpha)f(\gamma)f(\beta)$. So $S' \Rightarrow^\star_{G'} w$ and $w \in \mathcal{L}(G')$. □

Proposition 2.10

Let $G = (\Sigma, V, P, S)$ be a grammar and let V' be any set with $|V| = |V'|$ and $V' \cap \Sigma = \varnothing$. Then there are P' and S' with $G' = (\Sigma, V', P', S')$ equivalent to G.

Proof. Choose a bijection $f : V \to V'$ and extend it to Ω by the identity on Σ, so that (i) and (iii) hold. Set $S' = f(S)$ and $P' = \{f(\alpha) \to f(\beta) \mid \alpha \to \beta \in P\}$, so that (ii) and (iv) hold. Then $G \cong G'$, so the two are equivalent. □

Proposition 2.11

For a fixed alphabet Σ, there are only countably many languages of the form $\mathcal{L}(G)$ for a grammar G.

Proof. Fix a finite set V disjoint from Σ. There are only countably many rewrite systems on $\Omega = \Sigma \cup V$, and only finitely many choices of start symbol, so the collection $\mathcal{G}_V$ of grammars with this variable set is countable, and hence so is $\mathcal{L}_V = \{\mathcal{L}(G) \mid G \in \mathcal{G}_V\}$.

By Proposition 2.10, $\mathcal{L}_V$ depends only on $|V|$, so we may write $\mathcal{L}_n$ for the value at any n-element V, and every language generated by a grammar lies in some $\mathcal{L}_n$. Then $\bigcup_{n>0} \mathcal{L}_n$ is a countable union of countable sets and is therefore countable. □

Combining this with Corollary 1.5, almost every language is generated by no grammar at all. Grammars are nevertheless the right thing to study, because the languages they generate are exactly the ones we can hope to compute with; we shall see in Section 8 that they are precisely the *computably enumerable* languages.

§2.4 The Chomsky Hierarchy

Restricting the *shape* of the allowed production rules restricts the languages that can be generated, and Chomsky's classification is obtained by imposing four increasingly severe such restrictions.

Definition 2.12 (Types of production rule)

Let $\alpha \to \beta$ be a production rule. We call it

(i) **noncontracting** if $|\alpha| \leq |\beta|$;

(ii) **context-sensitive** if there are $A \in V$, $\gamma, \delta \in \Omega^\star$ and $\eta \in \Omega^\star$ with $|\eta| > 0$ such that $\alpha = \gamma A \delta$ and $\beta = \gamma \eta \delta$;

(iii) **context-free** if $\alpha = A \in V$ and $|\beta| > 0$;

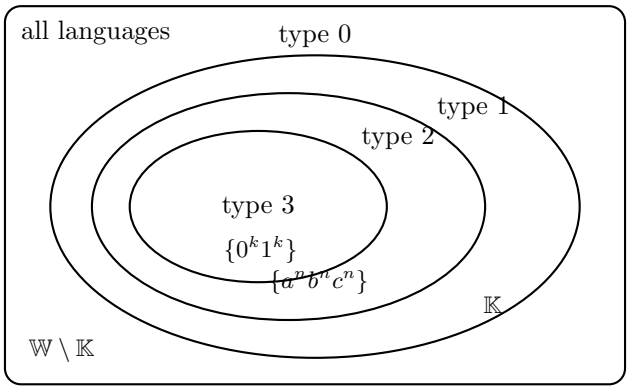
(iv) **regular** if $\alpha = A \in V$ and β is either a or aB for some $a \in \Sigma$ and $B \in V$.

A grammar has one of these properties if all of its rules do, and a language has one of these properties if it is generated by some grammar which has it.

A context-sensitive rule may replace A by η, but only when A occurs in the context γ_δ; the surrounding material is not allowed to change. A context-free rule may replace A by β wherever it occurs. A regular rule may only replace a variable by a letter, or by a letter followed by a single variable, so that a derivation grows strictly from left to right.

Immediately from the definitions, regular $\Rightarrow$ context-free $\Rightarrow$ context-sensitive $\Rightarrow$ non-contracting. (For the middle implication, take $\gamma = \delta = \varepsilon$.) In Chomsky's numbering, a language L is

- **type 0** if $L = \mathcal{L}(G)$ for some grammar G;
- **type 1** if $L = \mathcal{L}(G)$ for some context-sensitive G;
- **type 2** if $L = \mathcal{L}(G)$ for some context-free G;
- **type 3** if $L = \mathcal{L}(G)$ for some regular G.



The picture is the one to keep in mind, and proving that the inclusions are all strict – that the sample languages really do sit where they are drawn – occupies a large part of the course. We will show that $\{0^k1^k \mid k > 0\}$ is context-free but not regular, and that $\{a^nb^nc^n \mid n > 0\}$ is context-sensitive but not context-free; the halting set $\mathbb{K}$ and its complement will be dealt with in Section 8.

The following theorem of Chomsky says that two of our four conditions define the same class of languages, even though they do not define the same class of grammars. We shall not prove it.

Theorem 2.13 (Chomsky)

A language is noncontracting if and only if it is context-sensitive.

§2.5 Decision Problems

Alongside the question 'which languages arise?' we ask 'what can we work out about them?'. Three questions will accompany us throughout.

Definition 2.14 (The three decision problems)

(i) The **word problem** takes as input a grammar G and a word w, and asks whether $w \in \mathcal{L}(G)$.

(ii) The **emptiness problem** takes as input a grammar G and asks whether $\mathcal{L}(G) = \varnothing$.

(iii) The **equivalence problem** takes as input grammars G and G' and asks whether $\mathcal{L}(G) = \mathcal{L}(G')$.

A problem is **solvable** if there is an algorithm giving the correct answer for every input, and **unsolvable** otherwise.

For the moment 'algorithm' is informal; in Section 8.7 we will replace it by a precise definition and revisit all three problems. It will turn out that all three are unsolvable for grammars in general. Lower down the hierarchy things are better, and we can already deal with the word problem.

Lemma 2.15

Let G be a noncontracting grammar and let $w \in \mathbb{W}$. There is a bound N, depending only on $|w|$ and $|\Omega|$, such that $w \in \mathcal{L}(G)$ if and only if w has a G-derivation of length at most N.

Proof. Let $S = \sigma_0 \Rightarrow_G \cdots \Rightarrow_G \sigma_n = w$ be a derivation of minimal length. As G is noncontracting, the sequence of lengths $1 = |\sigma_0| \leq |\sigma_1| \leq \cdots \leq |\sigma_n| = |w|$ is nondecreasing. If $\sigma_r = \sigma_s$ for some $r < s$ then deleting $\sigma_{r+1}, \ldots, \sigma_s$ leaves a shorter derivation, so by minimality all the σ_i are distinct. But every σ_i is a string over Ω of length at most $|w|$, and there are only

$$N = \sum_{\ell=1}^{|w|} |\Omega|^\ell$$

of these, so $n \leq N$ by pigeonhole. □

Corollary 2.16

The word problem is solvable for noncontracting, context-sensitive, context-free and regular grammars.

Proof. Given G and w, compute N as above. There are only finitely many derivations of length at most N, since at each step only finitely many rules can be applied at finitely many positions, and they can be enumerated; check each one. □

This is a good illustration of the difference between solvability and practicality. The bound N is astronomically large, and the algorithm above is of no practical use whatsoever; but solvability is a statement about existence, and existence is what we get.

§2.6 Closure Properties

The other standard question about a class of languages is which operations it is closed under. Given languages L and M over Σ, the operations of interest are

$$LM, \quad L \cup M, \quad L \cap M, \quad L \setminus M, \quad \mathbb{W}^+ \setminus L.$$

We use $\mathbb{W}^+ \setminus L$ rather than $\mathbb{W} \setminus L$ for the complement, because (as we check below) a noncontracting grammar can never produce ε, so no class below type 0 could possibly be closed under an operation that introduces the empty word.

Some closure properties come for free from others: for instance, closure under complement and union implies closure under intersection, by De Morgan.

Definition 2.17 (Union and concatenation grammars)

Let $G = (\Sigma, V, P, S)$ and $G' = (\Sigma, V', P', S')$ be grammars, and let T be a variable in neither V nor V'. The **concatenation grammar** is

$$H = (\Sigma,\, V \cup V' \cup \{T\},\, P \cup P' \cup \{T \to SS'\},\, T),$$

and the **union grammar** is

$$H' = (\Sigma,\, V \cup V' \cup \{T\},\, P \cup P' \cup \{T \to S,\, T \to S'\},\, T).$$

It is clear that $\mathcal{L}(G)\mathcal{L}(G') \subseteq \mathcal{L}(H)$ and $\mathcal{L}(G) \cup \mathcal{L}(G') \subseteq \mathcal{L}(H')$. The reverse inclusions, however, are *not* automatic. Even after relabelling so that $V \cap V' = \varnothing$, the two rule sets can interact: a rule of P whose left-hand side contains letters can fire on a string produced by P'. The way to rule this out is to arrange that no rule mentions a letter on its left.

Definition 2.18 (Variable-based)

A rule $\alpha \to \beta$ is **variable-based** if every symbol occurring in α is a variable, and a grammar is variable-based if all of its rules are.

Regular and context-free grammars are automatically variable-based, since their rules have $\alpha = A \in V$. Context-sensitive grammars need not be.

Lemma 2.19

Every grammar is equivalent to a variable-based grammar. Moreover, if the original grammar is context-free, context-sensitive or noncontracting, then so is the new one.

Proof. Let $G = (\Sigma, V, P, S)$. For each letter $a \in \Sigma$ introduce a fresh variable X_a, and let $X \colon \Omega \to \Omega$ send $a \mapsto X_a$ for $a \in \Sigma$ and fix V pointwise; lift X to $\Omega^\star$. Put

$$P' = \{X(\alpha) \to X(\beta) \mid \alpha \to \beta \in P\}, \qquad P'' = \{X_a \to a \mid a \in \Sigma\},$$

and let $G' = (\Sigma, V \cup \{X_a \mid a \in \Sigma\}, P' \cup P'', S)$. Every rule of P' has only variables on the left by construction, and every rule of P'' has a single variable on the left, so G' is variable-based. The added rules $X_a \to a$ are regular, so they preserve context-freeness, context-sensitivity and noncontraction.

It remains to check $\mathcal{L}(G) = \mathcal{L}(G')$. Applying X to a G-derivation of w gives a G'-derivation of $X(w)$, and then applying the rules $X_a \to a$ once for each letter of w gives w; so $\mathcal{L}(G) \subseteq \mathcal{L}(G')$.

Conversely let $S = \sigma_0 \Rightarrow_{G'} \cdots \Rightarrow_{G'} \sigma_m = w$ be a G'-derivation. Apply X to every term to get a sequence $\tau_0, \dots, \tau_m$ with $\tau_0 = S$ and $\tau_m = X(w)$. If the step $\sigma_i \Rightarrow_{G'} \sigma_{i+1}$ used a rule $X(\alpha) \to X(\beta)$, then since X fixes such strings we still have $\tau_i \Rightarrow_{G'} \tau_{i+1}$ by the same rule, which is a rule of G after erasing X. If instead it used a rule $X_a \to a$, then $\tau_i = \tau_{i+1}$, and we delete the repetition. Deleting all such repetitions leaves a genuine G-derivation of w (after erasing X throughout, which is legitimate since no string in the resulting sequence contains any X_a). Hence $w \in \mathcal{L}(G)$. □

Theorem 2.20

Let G and G' be variable-based grammars over Σ with $V \cap V' = \varnothing$. Then

$$\mathcal{L}(H) = \mathcal{L}(G)\mathcal{L}(G'), \qquad \mathcal{L}(H') = \mathcal{L}(G) \cup \mathcal{L}(G').$$

Consequently the classes of context-free, context-sensitive and noncontracting languages are closed under union and concatenation, and the class of regular languages is closed under union.

Proof. One inclusion in each case has already been observed. For the other, consider a derivation in H. The first step must be $T \to SS'$, since T occurs in no other rule's right-hand side and appears nowhere else. Thereafter every rule applied has a variable-only left-hand side, so it lies entirely within the V-part or entirely within the V'-part of the string, these being disjoint. Thus the derivation splits into a G-derivation acting on the first component and a G'-derivation acting on the second, and any word produced is of the form vw with $v \in \mathcal{L}(G)$ and $w \in \mathcal{L}(G')$. The argument for H' is the same, the first step being $T \to S$ or $T \to S'$.

For the closure statements, given two grammars of the relevant type we first pass to equivalent variable-based grammars with disjoint variable sets, using Lemma 2.19 and Proposition 2.10, and then apply the above. The union grammar of two regular grammars is not regular, because of the rules $T \to S$; we shall prove closure of the

regular languages under union and concatenation by other means in Section 3. □

§2.7 The Empty Word

Remark. No noncontracting grammar produces ε: every string in a derivation from S has length at least $|S| = 1$. In particular no type 1, 2 or 3 language contains the empty word, which is why $\mathbb{W}^+$ rather than $\mathbb{W}$ is the correct ambient set for those classes.

One might hope simply to add the rule $S \to \varepsilon$ to put ε into a language, but this can have side effects: if S occurs on the right-hand side of some rule then the new rule can fire in the middle of a derivation and delete a symbol that was doing work.

Definition 2.21

A rule $\alpha \to \beta$ is **S-safe** if S does not occur in β. A grammar is **ε-adequate** if all of its rules are S-safe.

If G is ε-adequate then S can only ever appear as the very first string of a derivation, so adding $S \to \varepsilon$ produces a grammar for $\mathcal{L}(G) \cup \{\varepsilon\}$ and nothing else. Moreover every grammar is equivalent to an ε-adequate one: replace $G = (\Sigma, V, P, S)$ by

$$G' = (\Sigma,\, V \cup \{T\},\, P \cup \{T \to S\},\, T)$$

for a fresh variable T. This will be used without comment whenever it is convenient.

§3 Regular Languages

The lowest level of the hierarchy is the most thoroughly understood, and it is where we start. We will see that regular languages admit four completely different descriptions – by grammars, by deterministic automata, by nondeterministic automata, and by regular expressions – and that all four are effectively interchangeable. We will also obtain a tool (the pumping lemma) for proving that a language is *not* regular, and a canonical minimal automaton for each regular language.

§3.1 Regular Derivations

Recall that a regular rule has the form $A \to a$ or $A \to aB$. We call the first kind **terminal** and the second kind **nonterminal**. The shape of regular derivations is extremely rigid, which is what makes the theory work.

Lemma 3.1

Let G be a regular grammar. If $S \Rightarrow_G^\star \alpha$ then $\alpha \in \mathbb{W} \cup \mathbb{W}V$; that is, α is a word, possibly followed by a single variable.

Proof. Induction on the length of the derivation. The empty derivation gives $\alpha = S = \varepsilon S \in \mathbb{W}V$. Suppose $S \Rightarrow_G^\star \beta \Rightarrow_G \alpha$ with $\beta \in \mathbb{W} \cup \mathbb{W}V$. If $\beta \in \mathbb{W}$ then β contains no variable, but every rule rewrites a variable, so no step is possible; so $\beta = wA$ with $w \in \mathbb{W}$, $A \in V$. The applicable rules are $A \to a$ and $A \to aB$, giving $\alpha = wa \in \mathbb{W}$ or $\alpha = waB \in \mathbb{W}V$ respectively. □

Lemma 3.2

Let G be regular and $w \in \mathbb{W}$ with $S \Rightarrow_G^\star w$. Then every derivation of w from S has length exactly $|w|$, and consists of $|w| - 1$ nonterminal rules followed by a single terminal rule.

Proof. A nonterminal rule increases the length of the string by one and leaves the number of variables at one; a terminal rule leaves the length unchanged and reduces the number of variables from one to zero. Starting from S, of length 1 with one variable, we therefore need exactly $|w| - 1$ nonterminal steps to reach length $|w|$. Once the variable count reaches zero no further rule applies, so the terminal rule must come last, and there is exactly one of them. □

Note that the derivation is nonetheless not *unique*, since there may be several rules $A \to aB$ with the same A and a. This is precisely the nondeterminism that we will have to deal with below.

As a first application, we can now handle concatenation, which Theorem 2.20 did not give us for regular languages.

Lemma 3.3

Regular languages are closed under concatenation.

Proof. Let $G = (\Sigma, V, P, S)$ and $G' = (\Sigma, V', P', S')$ be regular, with $V \cap V' = \varnothing$ after relabelling. Let $P^\star$ be obtained from P by replacing every terminal rule $A \to a$ by the nonterminal rule $A \to aS'$, and set $H = (\Sigma, V \cup V', P^\star \cup P', S)$. This is regular. We claim $\mathcal{L}(H) = \mathcal{L}(G)\mathcal{L}(G')$.

If $S \Rightarrow_G^\star v$ and $S' \Rightarrow_{G'}^\star w$, then running the G-derivation in H and using the modified last rule yields $S \Rightarrow_H^\star vS'$, and then the G'-derivation gives $S \Rightarrow_H^\star vw$.

Conversely suppose $S \Rightarrow_H^\star u$ with $u \in \mathbb{W}$. By Lemma 3.1 every intermediate string is $w_i X_i$ with $X_i \in V \cup V'$. Initially $X_0 = S \in V$. The only rules taking a variable of V to a variable of V' are the modified rules $A \to aS'$, and no rule of P' mentions a variable of V; so once the derivation enters V' it stays there. Splitting the derivation at the unique step where this happens gives $u = vw$ with $S \Rightarrow_G^\star v$ (using the original terminal rule in place of the modified one) and $S' \Rightarrow_{G'}^\star w$. □

§3.2 Deterministic Automata

We now come to the machine model. A deterministic automaton reads a word one letter at a time, from left to right, keeping track of nothing except which of finitely many states it is in.

Definition 3.4 (Deterministic automaton)

Let Σ be an alphabet. A **deterministic automaton** is a tuple $D = (\Sigma, Q, \delta, q_0, F)$ where Q is a finite set of **states**, $q_0 \in Q$ is the **start state**, $F \subseteq Q \setminus \{q_0\}$ is the set of **accept states**, and $\delta \colon Q \times \Sigma \to Q$ is the **transition function**.

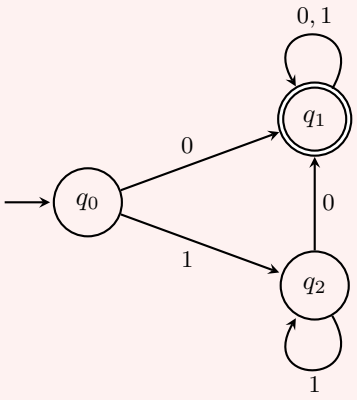
The requirement $q_0 \notin F$ is a convention forced on us by the fact that regular grammars

cannot generate ε; it will mean that ε never lies in the language of an automaton, matching the grammar side exactly. It is a harmless restriction, and we will return to it when we discuss the Kleene star.

We draw an automaton as a labelled directed graph: one node per state, drawn as a circle, or as a double circle if the state is accepting; an unlabelled incoming arrow marks the start state; and for each q and a there is an arrow from q to $\delta(q, a)$ labelled a. Every node therefore has exactly $|\Sigma|$ outgoing arrows, though several may be drawn as one arrow with several labels.

Example 3.5

Let $\Sigma = \{0, 1\}$ and consider the following automaton.



Reading 110 we pass $q_0 \to q_2 \to q_2 \to q_1$ and finish in an accept state; reading 111 we pass $q_0 \to q_2 \to q_2 \to q_2$ and do not. We shall verify below that this machine accepts exactly the words containing at least one 0.

To say precisely what a machine accepts we extend δ from letters to words.

Definition 3.6 (Language of an automaton)

Define $\hat{\delta}\colon Q \times \mathbb{W} \to Q$ by recursion on the length of the word:

$$\hat{\delta}(q, \varepsilon) = q, \qquad \hat{\delta}(q, aw) = \hat{\delta}(\delta(q, a), w).$$

The **language accepted by** D is

$$\mathcal{L}(D) = \{w \in \mathbb{W} \mid \hat{\delta}(q_0, w) \in F\}.$$

The **state sequence** of D on input $w = a_0 a_1 \cdots a_{n-1}$ is the sequence $q_0, q_1, \ldots, q_n$ with $q_{i+1} = \delta(q_i, a_i)$; it is uniquely determined and has length $|w| + 1$.

Note that $\varepsilon \notin \mathcal{L}(D)$ for every D, since $\hat{\delta}(q_0, \varepsilon) = q_0 \notin F$.

Example 3.7

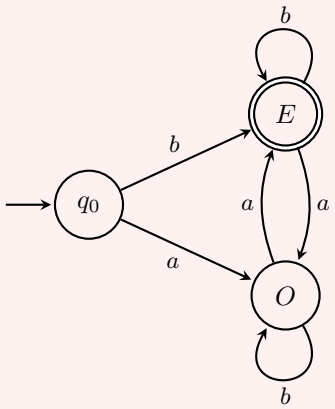
We verify the claim of Example 3.5: $\mathcal{L}(D)$ is the set of words containing at least one 0.

Every 0-labelled arrow in the diagram leads to q_1, and both arrows out of q_1 return

to q_1. So if w contains a 0, then after reading the first 0 the machine is in q_1 and stays there, and w is accepted. If w contains no 0, then $w = 1^k$; by induction $\hat{\delta}(q_0, 1^k) = q_2$ for $k \geq 1$ and $= q_0$ for $k = 0$, and neither is accepting. So w is rejected.

Example 3.8

Let $\Sigma = \{a, b\}$. The following automaton accepts the nonempty words containing an even number of a's.



The states E and O record the parity of the number of a's read so far, and q_0 is a copy of E which exists only to keep ε out of the language. Thus b is accepted ($q_0 \to E$), aa is accepted ($q_0 \to O \to E$), and a is rejected ($q_0 \to O$), as is ε.

The last example is slightly unsatisfying: the states q_0 and E do the same job. We will make this observation precise in Section 3.8, where we show that every automaton has a canonical smallest equivalent form. To do that we need a notion of map between automata.

Definition 3.9 (Homomorphism)

Let $D = (\Sigma, Q, \delta, q_0, F)$ and $D' = (\Sigma, Q', \delta', q_0', F')$. A function $f\colon Q \to Q'$ is a **homomorphism** if

(i) $\delta'(f(q), a) = f(\delta(q, a))$ for all $q \in Q$, $a \in \Sigma$;

(ii) $f(q_0) = q_0'$;

(iii) $q \in F$ if and only if $f(q) \in F'$.

A bijective homomorphism is an **isomorphism**; its inverse is then a homomorphism too.

Condition (i) says that f commutes with a single step, and a trivial induction upgrades this to $\hat{\delta}'(f(q), w) = f(\hat{\delta}(q, w))$ for all words w.

Proposition 3.10

If there is a homomorphism $f\colon D \to D'$ then $\mathcal{L}(D) = \mathcal{L}(D')$.

Proof. For any word w we have

$$\hat{\delta}(q_0, w) \in F \iff f(\hat{\delta}(q_0, w)) \in F' \iff \hat{\delta}'(q_0', w) \in F'$$

using (iii), then (i) and (ii). □

So a homomorphism, even a very non-injective and non-surjective one, preserves the language. This tells us what failures of bijectivity mean. If f is not surjective, the missing states of D' are not reachable from q_0'. If f is not injective, say $f(p) = f(q)$ with $p \neq q$, then for every word w

$$f(\hat{\delta}(p, w)) = \hat{\delta}'(f(p), w) = \hat{\delta}'(f(q), w) = f(\hat{\delta}(q, w)),$$

so $\hat{\delta}(p, w) \in F$ if and only if $\hat{\delta}(q, w) \in F$: the states p and q cannot be told apart by any word. Both phenomena are forms of redundancy, and eliminating them is exactly what minimisation does.

§3.3 From Automata to Grammars

Theorem 3.11

If D is a deterministic automaton then $\mathcal{L}(D)$ is regular.

Proof. Let $D = (\Sigma, Q, \delta, q_0, F)$ and define a grammar $G = (\Sigma, Q, P, q_0)$ with variable set Q and

$$P = \{p \to aq \mid \delta(p, a) = q\} \cup \{p \to a \mid \delta(p, a) \in F\}.$$

Every rule is regular. Let $w = a_0 a_1 \cdots a_n$ be a nonempty word, and let $q_0, q_1, \ldots, q_{n+1}$ be its state sequence. Then

$$\begin{aligned} w \in \mathcal{L}(D) &\iff q_{n+1} \in F \\ &\iff q_i \to a_i q_{i+1} \in P \text{ for } i < n, \text{ and } q_n \to a_n \in P \\ &\iff q_0 \Rightarrow_G a_0 q_1 \Rightarrow_G \cdots \Rightarrow_G a_0 \cdots a_{n-1} q_n \Rightarrow_G w, \end{aligned}$$

where the middle equivalence holds because the state sequence is uniquely determined by w. By Lemma 3.2 every derivation of w in G has this shape, so the last line holds if and only if $w \in \mathcal{L}(G)$. Finally ε lies in neither language. □

We would like the converse. The obvious attempt – take $Q = V$ and read off δ from the rules – fails immediately, because a regular grammar may contain both $A \to aB$ and $A \to aB'$, and then there is no single value for $\delta(A, a)$. The standard remedy is to allow the machine to be in a *set* of states at once.

§3.4 Nondeterministic Automata and the Subset Construction

Definition 3.12 (Nondeterministic automaton)

A **nondeterministic automaton** is a tuple $N = (\Sigma, Q, \delta, q_0, F)$ exactly as before, except that

$$\delta \colon Q \times \Sigma \to \mathcal{P}(Q).$$

We read $\delta(q, a)$ as the set of states the machine *may* move to on reading a in state q;

if $\delta(q,a) = \varnothing$ then the computation gets stuck. The graphical convention is unchanged, except that a node may now have several, or no, outgoing arrows for a given letter.

Definition 3.13

For a nondeterministic automaton define $\hat{\delta}\colon Q \times \mathbb{W} \to \mathcal{P}(Q)$ by

$$\hat{\delta}(q,\varepsilon) = \{q\}, \qquad \hat{\delta}(q,wa) = \bigcup_{p \in \hat{\delta}(q,w)} \delta(p,a),$$

and set

$$\mathcal{L}(N) = \{w \in \mathbb{W} \mid \hat{\delta}(q_0,w) \cap F \neq \varnothing\}.$$

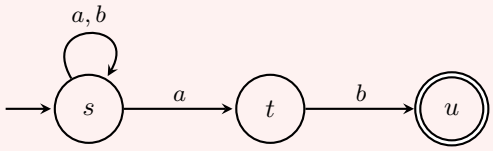
So N accepts w if *some* run on w ends in an accept state. A deterministic automaton is the special case in which every $\delta(q,a)$ is a singleton, and the two definitions of $\hat{\delta}$ then agree.

Example 3.14

Let $\Sigma = \{a,b\}$ and consider the nondeterministic automaton N with states s (start), t and u (accept), and

$$\delta(s,a) = \{s,t\}, \quad \delta(s,b) = \{s\}, \quad \delta(t,b) = \{u\},$$

all other values being $\varnothing$.



The machine guesses when it has reached the penultimate letter. It accepts exactly the words ending in ab: on input aab we get $\hat{\delta}(s,a) = \{s,t\}$, then $\hat{\delta}(s,aa) = \{s,t\}$, then $\hat{\delta}(s,aab) = \{s\} \cup \{u\} = \{s,u\}$, which meets F; whereas on input aba we get $\{s,t\}$, then $\{s,u\}$, then $\{s,t\}$, which does not.

The nondeterminism is only apparent, and this is the fundamental theorem about finite automata.

Theorem 3.15 (Subset construction)

Let N be a nondeterministic automaton. Then there is a deterministic automaton D with $\mathcal{L}(N) = \mathcal{L}(D)$.

Proof. Let $N = (\Sigma, Q, \delta, q_0, F)$ and define

$$D = (\Sigma,\ \mathcal{P}(Q),\ \Delta,\ \{q_0\},\ \mathcal{F}), \qquad \Delta(X,a) = \bigcup_{q \in X} \delta(q,a), \qquad \mathcal{F} = \{X \subseteq Q \mid X \cap F \neq \varnothing\}.$$

(If $\{q_0\} \in \mathcal{F}$, that is if $q_0 \in F$, then F was not a legal accept set for N in the first place; so the convention $\mathcal{F} \subseteq \mathcal{P}(Q) \setminus \{\{q_0\}\}$ holds automatically.)

Fix a word $w = a_0 \cdots a_{n-1}$. The state sequence of D on w is $X_0 = \{q_0\}$ and $X_{i+1} = \bigcup_{q \in X_i} \delta(q, a_i)$; the sequence of state-sets of N is $Y_0 = \{q_0\}$ and $Y_{i+1} = \bigcup_{q \in Y_i} \delta(q, a_i)$. These recursions are identical, so $X_i = Y_i$ for all i, whence

$$w \in \mathcal{L}(D) \iff X_n \cap F \neq \varnothing \iff \hat{\delta}(q_0, w) \cap F \neq \varnothing \iff w \in \mathcal{L}(N). \qquad \square$$

Remark. The construction turns an n-state machine into one with 2^n states, and this exponential blow-up cannot in general be avoided. In practice one only ever builds the subsets *reachable* from $\{q_0\}$, which is often far fewer.

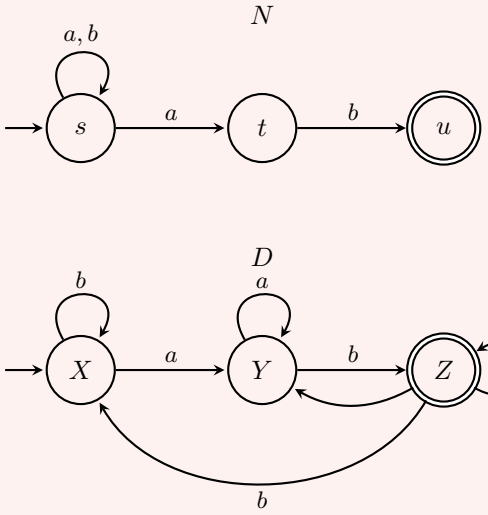
Example 3.16

Let us carry out the subset construction on the automaton N of Example 3.14. Starting from $\{s\}$ and closing off under Δ:

$$\Delta(\{s\}, a) = \{s, t\}, \quad \Delta(\{s\}, b) = \{s\}, \quad \Delta(\{s, t\}, a) = \{s, t\}, \quad \Delta(\{s, t\}, b) = \{s, u\},$$

$$\Delta(\{s, u\}, a) = \{s, t\}, \quad \Delta(\{s, u\}, b) = \{s\}.$$

Only three of the eight subsets are reachable, and only $\{s, u\}$ meets $F = \{u\}$. Writing $X = \{s\}$, $Y = \{s, t\}$, $Z = \{s, u\}$ we obtain the deterministic automaton on the right below.



Tracing *abab* through D: $X \to Y \to Z \to Y \to Z$, ending in the accept state, and indeed *abab* ends in *ab*. Tracing *abb*: $X \to Y \to Z \to X$, rejected, and indeed *abb* does not end in *ab*.

Theorem 3.17

If G is a regular grammar then there is a nondeterministic automaton N with $\mathcal{L}(G) = \mathcal{L}(N)$.

Proof. Let $G = (\Sigma, V, P, S)$ and let H be a fresh symbol, the **halt state**. Put

$Q = V \cup \{H\}$ and define $N = (\Sigma, Q, \delta, S, \{H\})$ where, for $A \in V$,

$$\delta(A,a) = \begin{cases} \{B \mid A \to aB \in P\} & \text{if } A \to a \notin P, \\ \{B \mid A \to aB \in P\} \cup \{H\} & \text{if } A \to a \in P, \end{cases}$$

and $\delta(H,a) = \varnothing$. Note $H \neq S$, so the convention $F \subseteq Q \setminus \{q_0\}$ holds.

Let $w = a_0 \cdots a_n$ be a nonempty word. By Lemma 3.2, $w \in \mathcal{L}(G)$ if and only if there are variables $S = A_0, A_1, \ldots, A_n$ with $A_i \to a_i A_{i+1} \in P$ for $i < n$ and $A_n \to a_n \in P$. By the definition of δ, this holds if and only if there are states with $A_{i+1} \in \delta(A_i, a_i)$ for $i < n$ and $H \in \delta(A_n, a_n)$, which holds if and only if $H \in \hat{\delta}(S, w)$, i.e. $w \in \mathcal{L}(N)$. Neither language contains ε. □

Assembling Theorems 3.11, 3.15 and 3.17, we have the following, which is the first genuinely satisfying theorem of the course.

Corollary 3.18

For a language L, the following are equivalent:

(i) L is regular, i.e. $L = \mathcal{L}(G)$ for a regular grammar G;

(ii) $L = \mathcal{L}(N)$ for a nondeterministic automaton N;

(iii) $L = \mathcal{L}(D)$ for a deterministic automaton D.

Moreover, each translation is effective: given one of the three objects we can compute the other two.

§3.5 The Pumping Lemma for Regular Languages

We now have three ways to show a language *is* regular. To show that one is not, the standard tool is the following. It expresses in combinatorial terms the fact that a machine with finitely many states must, on a long enough word, revisit a state, and can then be made to go round the resulting loop as often as we please.

Definition 3.19 (Regular pumping lemma)

A language L **satisfies the regular pumping lemma with pumping number** n if every $w \in L$ with $|w| \geq n$ can be written as $w = xyz$ with

$$|y| > 0, \qquad |xy| \leq n, \qquad xy^k z \in L \text{ for all } k \in \mathbb{N}.$$

We call y a **pump** for w.

Theorem 3.20 (Pumping lemma)

Every regular language satisfies the regular pumping lemma, with pumping number the number of states of any deterministic automaton accepting it.

Proof. Let $L = \mathcal{L}(D)$ with $D = (\Sigma, Q, \delta, q_0, F)$, and put $n = |Q|$. Let $w \in L$ with $|w| \geq n$, say $w = a_0 a_1 \cdots a_{n-1} v$.

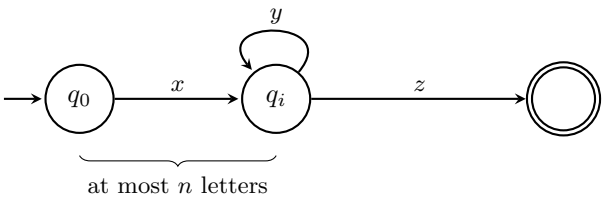
Let $q_0, q_1, \ldots, q_n$ be the states visited while reading $a_0 \cdots a_{n-1}$. This is a list of $n+1$ states drawn from a set of size n, so by pigeonhole there are $i < j \leq n$ with $q_i = q_j$. Put

$$x = a_0 \cdots a_{i-1}, \qquad y = a_i \cdots a_{j-1}, \qquad z = a_j \cdots a_{n-1} v,$$

so that $xyz = w$, $|y| = j - i > 0$ and $|xy| = j \leq n$.

Now $\hat{\delta}(q_0, x) = q_i$ and $\hat{\delta}(q_i, y) = q_j = q_i$, so by induction on k we get $\hat{\delta}(q_0, xy^k) = q_i$ for every $k \in \mathbb{N}$. Finally $\hat{\delta}(q_i, z) = \hat{\delta}(q_j, z) = \hat{\delta}(q_0, w) \in F$, so $\hat{\delta}(q_0, xy^k z) \in F$ and $xy^k z \in L$. □

The picture behind the proof is worth drawing: the accepting path through the machine, of length at least n, must contain a loop within its first n steps, and the loop can be traversed any number of times.



Example 3.21

The language $L = \{0^k 1^k \mid k > 0\}$ is not regular.

Suppose it were, with pumping number N. The word $w = 0^N 1^N$ lies in L and has length $2N \geq N$, so it can be written $w = xyz$ with $|xy| \leq N$ and $|y| = \ell > 0$. Since $|xy| \leq N$, the pump y lies within the first N letters of w, all of which are 0s; so $y = 0^\ell$. Pumping down with $k = 0$ gives $xz = 0^{N-\ell} 1^N \in L$, which is false since $\ell > 0$. So L is not regular.

We will see in Section 4 that $L = \{0^k 1^k\}$ *is* context-free, so this example already shows that the inclusion of type 3 in type 2 is strict.

Example 3.22

Fix $n > 0$ and let $L = \{0^n w \mid w \in \mathbb{W}\}$, the words beginning with n zeroes. This is regular: take $V = \{S, X_0, \ldots, X_{n-1}\}$ and

$$P = \{S \to 0X_0\} \cup \{X_i \to 0X_{i+1} \mid i \leq n-3\} \cup \{X_{n-2} \to 0X_{n-1},\ X_{n-2} \to 0\}$$
$$\cup \{X_{n-1} \to 0,\ X_{n-1} \to 1,\ X_{n-1} \to 0X_{n-1},\ X_{n-1} \to 1X_{n-1}\}.$$

But no deterministic automaton with at most n states accepts L. Indeed, such an automaton would give L pumping number n by Theorem 3.20, and then the word $0^n \in L$ of length n could be pumped down to a word with fewer than n zeroes, which is not in L.

It is essential to remember that the pumping lemma is a necessary but *not* a sufficient condition for regularity. The next example makes this vivid.

Example 3.23

Let $\Sigma = \{0, 1\}$. For a word w containing at least one 0, define the **tail** of w to be the number of 1s following the last 0. Given $X \subseteq \mathbb{N}$, let

$$L_X = \{w \in \mathbb{W}^+ \mid w \text{ contains no } 0\} \cup \{w \mid w \text{ has a tail lying in } X\}.$$

If $X \neq Y$ then $L_X \neq L_Y$: taking m in the symmetric difference, the word 01^m distinguishes them. So $X \mapsto L_X$ is injective, and since $\mathcal{P}(\mathbb{N})$ is uncountable while there are only countably many regular languages, some L_X is not regular.

Nevertheless *every* L_X satisfies the pumping lemma with pumping number 2. Let $w \in L_X$ with $|w| \geq 2$.

- If $w = 0z$, take $x = \varepsilon$, $y = 0$. Pumping up prepends more 0s, which changes neither the presence of a 0 nor the tail. Pumping down gives z: either z still contains a 0, in which case the tail is unchanged, or it does not, in which case $z \in 1^\star$ and again lies in L_X (note $z \neq \varepsilon$ as $|w| \geq 2$).
- If $w = 1z$, take $x = \varepsilon$, $y = 1$. If z contains no 0 then neither does $1^k z$, so all pumps lie in L_X. If z contains a 0 then the tail of $1^k z$ equals the tail of z for every k, so again all pumps lie in L_X.

Hence there are uncountably many languages satisfying the regular pumping lemma, and in particular the pumping lemma does not characterise the regular languages.

§3.6 Closure Properties

We have already proved closure under union (Theorem 2.20 together with the automaton translation, or directly) and concatenation (Lemma 3.3). Automata make the Boolean operations easy.

Proposition 3.24

The regular languages are closed under complement (relative to $\mathbb{W}^+$), intersection and difference.

Proof. Let $L = \mathcal{L}(D)$ with $D = (\Sigma, Q, \delta, q_0, F)$. We may assume that $\delta(q, a) \neq q_0$ for all q, a: if not, add a new start state q_0' with $\delta(q_0', a) = \delta(q_0, a)$ and make q_0 an ordinary state, accepting or not according to whether some word takes q_0' back to it and lands in F – more simply, take a disjoint copy q_0' of q_0 as the new start state, put $\delta(q_0', a) = \delta(q_0, a)$, and let q_0 inherit its accepting status from whether $q_0 \in F$, which is to say it is not accepting. Then no transition enters q_0', and the language is unchanged since $\hat{\delta}(q_0', w) = \hat{\delta}(q_0, w)$ for $w \neq \varepsilon$.

Now set $D' = (\Sigma, Q, \delta, q_0, Q \setminus (F \cup \{q_0\}))$. For a nonempty word w we have $\hat{\delta}(q_0, w) \neq q_0$, so $\hat{\delta}(q_0, w) \in Q \setminus (F \cup \{q_0\})$ if and only if $\hat{\delta}(q_0, w) \notin F$. Hence $\mathcal{L}(D') = \mathbb{W}^+ \setminus L$.

Intersection and difference now follow from De Morgan, since $L \cap M = \mathbb{W}^+ \setminus ((\mathbb{W}^+ \setminus L) \cup (\mathbb{W}^+ \setminus M))$ and $L \setminus M = L \cap (\mathbb{W}^+ \setminus M)$. □

There is a more direct construction for intersection and union which is worth knowing, because it is the one that fails for the higher classes.

Definition 3.25 (Product automaton)

Given $D = (\Sigma, Q, \delta, q_0, F)$ and $D' = (\Sigma, Q', \delta', q_0', F')$, the **product automaton** is

$$D \times D' = (\Sigma,\ Q \times Q',\ \delta \times \delta',\ (q_0, q_0'),\ F''), \qquad (\delta \times \delta')((q,q'),a) = (\delta(q,a), \delta'(q',a)),$$

where F'' is either $\{(q,q') \mid q \in F \text{ and } q' \in F'\}$ or $\{(q,q') \mid q \in F \text{ or } q' \in F'\}$.

An immediate induction gives $\widehat{(\delta \times \delta')}((q_0,q_0'),w) = (\hat{\delta}(q_0,w), \hat{\delta}'(q_0',w))$, so the first choice of F'' yields $\mathcal{L}(D) \cap \mathcal{L}(D')$ and the second yields $\mathcal{L}(D) \cup \mathcal{L}(D')$. In words: the product machine runs both automata side by side on the same input, which it can afford to do because each has only finitely many states. We will see in Section 4 that no such construction can exist for context-free languages.

§3.7 Regular Expressions and Kleene's Theorem

There is a fourth description of the regular languages, and it is the one familiar from programming.

Definition 3.26 (Kleene star)

For a language L, the **Kleene star** of L is

$$L^\star = \{w_1 w_2 \cdots w_k \mid k \in \mathbb{N},\ w_i \in L\},$$

the set of all concatenations of finitely many words of L; in particular $\varepsilon \in L^\star$, taking $k = 0$. The **Kleene plus** is $L^+ = \{w_1 \cdots w_k \mid k \geq 1,\ w_i \in L\}$, so that $L^\star = L^+ \cup \{\varepsilon\}$.

Definition 3.27 (Regular expression)

The **regular expressions** over Σ are defined inductively:

(i) $\varnothing$ is a regular expression;

(ii) ε is a regular expression;

(iii) a is a regular expression for each $a \in \Sigma$;

(iv) if R, S are regular expressions then so is $(R + S)$;

(v) if R, S are regular expressions then so is (RS);

(vi) if R is a regular expression then so are $R^\star$ and R^+,

and nothing else is. To each regular expression E we assign a language $\mathcal{L}(E)$ by the matching recursion:

$$\mathcal{L}(\varnothing) = \varnothing, \quad \mathcal{L}(\varepsilon) = \{\varepsilon\}, \quad \mathcal{L}(a) = \{a\},$$

$$\mathcal{L}(R{+}S) = \mathcal{L}(R) \cup \mathcal{L}(S), \quad \mathcal{L}(RS) = \mathcal{L}(R)\mathcal{L}(S), \quad \mathcal{L}(R^\star) = \mathcal{L}(R)^\star, \quad \mathcal{L}(R^+) = \mathcal{L}(R)^+.$$

We drop parentheses when no ambiguity results, giving concatenation a higher binding power than $+$, so that $RS + T$ means $((RS) + T)$.

The only irritation is the empty word, which regular expressions can produce and regular

grammars cannot. We deal with it by a definition.

Definition 3.28

A language L is **essentially regular** if L or $L \setminus \{\varepsilon\}$ is regular; equivalently, if there is a regular L' with $L = L'$ or $L = L' \cup \{\varepsilon\}$.

Theorem 3.29

If E is a regular expression then $\mathcal{L}(E)$ is essentially regular.

Proof. We induct on the structure of E. The base cases are clear: $\varnothing$ and $\{a\}$ are regular (take $P = \varnothing$ and $P = \{S \to a\}$ respectively) and $\{\varepsilon\} = \varnothing \cup \{\varepsilon\}$ is essentially regular.

For the induction step it is enough to show that essentially regular languages are closed under union, concatenation and Kleene plus, since $\mathcal{L}(E^\star) = \mathcal{L}(E + E^+)$.

Union and concatenation of regular languages are regular by Theorem 2.20 and Lemma 3.3, and the extension to essentially regular languages is a short case analysis: for instance $(L \cup \{\varepsilon\})(M \cup \{\varepsilon\}) = LM \cup L \cup M \cup \{\varepsilon\}$, and $LM \cup L \cup M$ is regular.

So let $G = (\Sigma, V, P, S)$ be regular; we must show $\mathcal{L}(G)^+$ is regular. We may assume G is ε-adequate, so S never occurs on the right of a rule. Let

$$P^+ = P \cup \{A \to aS \mid A \to a \in P\}, \qquad G^+ = (\Sigma, V, P^+, S).$$

This is regular; we claim $\mathcal{L}(G^+) = \mathcal{L}(G)^+$.

($\supseteq$) Let $w = w_0 w_1 \cdots w_n$ with each $w_i \in \mathcal{L}(G)$. Induct on n. For $n = 0$ any G-derivation is a G^+-derivation. For $n > 0$, by induction $S \Rightarrow^\star_{G^+} w_0 \cdots w_{n-1}$; this derivation ends with a terminal rule $A \to a$, and replacing that final step by $A \to aS$ gives $S \Rightarrow^\star_{G^+} w_0 \cdots w_{n-1} S$. Appending a G-derivation of w_n gives w.

($\subseteq$) Let $S \Rightarrow^\star_{G^+} w$ and let $n \geq 1$ be the number of strings in the derivation in which S occurs (the first one always does). Induct on n. If $n = 1$ then S occurs only at the start, so no added rule was used and $S \Rightarrow^\star_G w$, giving $w \in \mathcal{L}(G) \subseteq \mathcal{L}(G)^+$. If $n > 1$, write the derivation as $S \Rightarrow^\star_{G^+} vS \Rightarrow^\star_{G^+} w$ where vS is the *last* string containing S. The step producing vS used an added rule $A \to aS$; replacing it by $A \to a$ gives a derivation $S \Rightarrow^\star_{G^+} v$ with $n - 1$ occurrences of S, so $v \in \mathcal{L}(G)^+$ by induction. From vS onward S never reappears, so no added rule fires, and $w = vw'$ with $S \Rightarrow^\star_G w'$, i.e. $w' \in \mathcal{L}(G)$. Hence $w \in \mathcal{L}(G)^+$. □

The converse is Kleene's theorem proper, and it is usually proved by an algorithm which eliminates the states of an automaton one at a time. We give the closely related dynamic-programming version.

Theorem 3.30 (Kleene)

Let D be a deterministic automaton. Then there is a regular expression E with $\mathcal{L}(E) = \mathcal{L}(D)$. Consequently a language is essentially regular if and only if it is $\mathcal{L}(E)$ for some regular expression E.

Proof. Write $Q = \{q_0, q_1, \dots, q_{m-1}\}$. For $i, j < m$ and $0 \le k \le m$, let

$$R^k_{i,j} = \{w \in \mathbb{W} \mid \hat{\delta}(q_i, w) = q_j \text{ and every state strictly inside the run lies in } \{q_0, \dots, q_{k-1}\}\},$$

where 'strictly inside' means excluding the first and last states of the run. We show by induction on k that each $R^k_{i,j}$ is $\mathcal{L}(E)$ for a regular expression $E^k_{i,j}$.

For $k = 0$ no intermediate states are allowed, so $R^0_{i,j}$ consists of ε (if $i = j$) together with the letters a with $\delta(q_i, a) = q_j$. This is a finite set of words of length at most 1, hence is $\mathcal{L}(E)$ for a finite sum of the base expressions.

For the induction step, a run from q_i to q_j through $\{q_0, \dots, q_k\}$ either avoids q_k altogether, or visits q_k; in the latter case, splitting at the first and last visits to q_k decomposes the word as a run $q_i \to q_k$ avoiding q_k internally, then finitely many runs $q_k \to q_k$ avoiding q_k internally, then a run $q_k \to q_j$ avoiding q_k internally. Hence

$$R^{k+1}_{i,j} = R^k_{i,j} \cup R^k_{i,k} \left(R^k_{k,k}\right)^\star R^k_{k,j},$$

and we may take $E^{k+1}_{i,j} = E^k_{i,j} + E^k_{i,k}(E^k_{k,k})^\star E^k_{k,j}$.

Finally $\mathcal{L}(D) = \bigcup_{q_j \in F} R^m_{0,j}$, so we take $E = \sum_{q_j \in F} E^m_{0,j}$. Since $q_0 \notin F$, the word ε does not lie in this language, so the resulting expression describes $\mathcal{L}(D)$ exactly.

For the last sentence, combine this with Theorem 3.29: if L is essentially regular then L is $\mathcal{L}(E)$ or $\mathcal{L}(E + \varepsilon)$ for a suitable E. □

Example 3.31

The automaton of Example 3.5 accepts the words containing at least one 0, and indeed

$$\mathcal{L}(D) = \mathcal{L}\big(1^\star 0(0+1)^\star\big).$$

The automaton of Example 3.16 accepts $\mathcal{L}((a+b)^\star ab)$.

§3.8 Minimisation

We saw in Example 3.8 an automaton with two states doing the same job. We now make this precise and show that every regular language has a canonical smallest automaton.

Definition 3.32

A state q is **accessible** if $q = \hat{\delta}(q_0, w)$ for some $w \in \mathbb{W}$, and **inaccessible** otherwise.

States q, q' are **distinguished by** w if exactly one of $\hat{\delta}(q, w)$, $\hat{\delta}(q', w)$ lies in F. They are **distinguishable** if some word distinguishes them, and **indistinguishable** otherwise; we write $q \sim q'$ in the latter case.

D is **irreducible** if every state is accessible and every pair of distinct states is distinguishable.

Suppose $f\colon D \to D'$ is a homomorphism. If p, q are distinguishable by w then, since f commutes with $\hat{\delta}$ and preserves acceptance, $f(p)$ and $f(q)$ are distinguished by w, so $f(p) \neq f(q)$. And if $q' \in Q'$ is accessible, say $q' = \hat{\delta}'(q'_0, w)$, then $q' = f(\hat{\delta}(q_0, w))$ lies in

the image of f. Hence:

Proposition 3.33

Any homomorphism between irreducible automata is an isomorphism.

Clearly $\sim$ is an equivalence relation on Q; write $[q]$ for the class of q.

Definition 3.34 (Quotient automaton)

Define

$$D/\!\sim \;=\; \big(\Sigma,\; Q/\!\sim,\; [\delta],\; [q_0],\; [F]\big), \qquad [\delta]([q],a) = [\delta(q,a)], \qquad [F] = \{[q] \mid q \in F\}.$$

Two checks are needed. First, being accepting is a class property: if $q \sim q'$ then ε does not distinguish them, so $q \in F \iff q' \in F$; in particular $[F]$ really is a set of classes, and $[q_0] \notin [F]$ since $q_0 \notin F$. Second, $[\delta]$ is well defined: if $q \sim q'$ but $\delta(q,a) \not\sim \delta(q',a)$, witnessed by a word w, then aw would distinguish q from q'.

Lemma 3.35

The quotient map $q \mapsto [q]$ is a surjective homomorphism, so $\mathcal{L}(D) = \mathcal{L}(D/\!\sim)$. Moreover distinct states of $D/\!\sim$ are distinguishable, and if every state of D is accessible then so is every state of $D/\!\sim$.

Proof. That the quotient map is a homomorphism is exactly the two checks above; it is surjective by construction, so accessibility is inherited and Proposition 3.10 gives the languages agree.

An easy induction gives $[\hat\delta]([q],w) = [\hat\delta(q,w)]$. Now suppose $[q] \neq [q']$, so $q \not\sim q'$; take w distinguishing them, say $\hat\delta(q,w) \in F$ and $\hat\delta(q',w) \notin F$. Then $[\hat\delta]([q],w) = [\hat\delta(q,w)] \in [F]$ while $[\hat\delta]([q'],w) \notin [F]$, so w distinguishes $[q]$ from $[q']$. □

Theorem 3.36

For every deterministic automaton D there is an irreducible deterministic automaton I with $\mathcal{L}(D) = \mathcal{L}(I)$ and at most as many states.

Proof. Let $A \subseteq Q$ be the set of accessible states; note $q_0 \in A$ and δ restricts to A. Put $D^\star = (\Sigma, A, \delta\restriction_{A\times\Sigma}, q_0, F \cap A)$. The inclusion $A \hookrightarrow Q$ is a homomorphism, so $\mathcal{L}(D^\star) = \mathcal{L}(D)$, and every state of $D^\star$ is accessible. Now take $I = D^\star/\!\sim$ and apply Lemma 3.35. □

Theorem 3.37

If I and I' are irreducible with $\mathcal{L}(I) = \mathcal{L}(I')$, then $I \cong I'$.

Proof. By Proposition 3.33 it suffices to construct a homomorphism $I \to I'$. Write $I = (\Sigma, Q, \delta, q_0, F)$ and $I' = (\Sigma, Q', \delta', q_0', F')$ with $Q \cap Q' = \varnothing$. Extend $\sim$ to $Q \cup Q'$ by declaring $q \sim q'$ (for $q \in Q$, $q' \in Q'$) when for all w, $\hat\delta(q,w) \in F \iff \hat\delta'(q',w) \in F'$;

this is again an equivalence relation. Since the languages agree, $q_0 \sim q_0'$.

Every $q \in Q$ is $\sim$-equivalent to some $q' \in Q'$. Let $\mathrm{sp}(q)$ be the length of a shortest word w with $\hat{\delta}(q_0, w) = q$, which exists by accessibility. Induct on $\mathrm{sp}(q)$. If $\mathrm{sp}(q) = 0$ then $q = q_0 \sim q_0'$. If $\mathrm{sp}(q) = k+1$, choose p and a with $\delta(p, a) = q$ and $\mathrm{sp}(p) = k$; by induction pick $p' \sim p$ and set $q' = \delta'(p', a)$. If w is any word then $\hat{\delta}(q, w) = \hat{\delta}(p, aw)$ and $\hat{\delta}'(q', w) = \hat{\delta}'(p', aw)$, so $p \sim p'$ gives $q \sim q'$.

The witness is unique. If $q' \sim q \sim p'$ with $q', p' \in Q'$ then $q' \sim p'$ by transitivity, and irreducibility of I' forces $q' = p'$.

So there is a well-defined $f : Q \to Q'$ with $q \sim f(q)$. Then $f(q_0) = q_0'$; taking $w = \varepsilon$ in the definition of $\sim$ gives $q \in F \iff f(q) \in F'$; and for any a, the computation above shows $\delta(q, a) \sim \delta'(f(q), a)$, so $f(\delta(q, a)) = \delta'(f(q), a)$ by uniqueness. Thus f is a homomorphism. □

Corollary 3.38

Every regular language is accepted by an irreducible automaton, unique up to isomorphism, and this automaton has fewer states than any other automaton accepting the language.

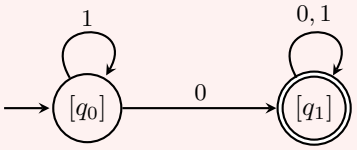
Proof. Existence and uniqueness are Theorems 3.36 and 3.37. If D is any automaton for L, Theorem 3.36 produces an irreducible I with at most $|Q|$ states, and I is isomorphic to the canonical one. □

Example 3.39

Return to Example 3.5. Is that automaton irreducible? All three states are accessible. The state q_1 is distinguished from the others by ε. But for any word w,

$$\hat{\delta}(q_0, w) \in F \iff w \text{ contains a } 0 \iff \hat{\delta}(q_2, w) \in F,$$

so $q_0 \sim q_2$ and the automaton is not irreducible. Quotienting merges them, and we obtain the two-state machine



which is irreducible. Notice that this still satisfies our convention that the start state is not accepting, as it must, since ε contains no 0. Applying the same analysis to Example 3.8 merges q_0 with E and produces the expected two-state parity machine.

§3.9 The Myhill–Nerode Theorem

Where do the states of the minimal automaton come from? The answer is that they are determined by the language alone, with no reference to any machine.

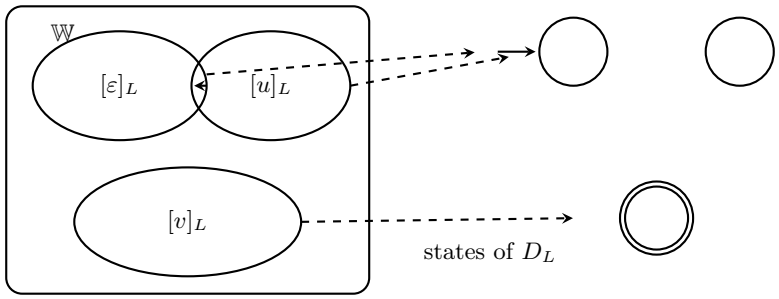
Definition 3.40 (Nerode equivalence)

Let $L \subseteq \mathbb{W}$ be any language. For $u, v \in \mathbb{W}$ write

$$u \sim_L v \quad \text{if and only if} \quad \text{for all } w \in \mathbb{W},\ uw \in L \iff vw \in L.$$

This is an equivalence relation on $\mathbb{W}$; write $[u]_L$ for the class of u and $\mathbb{W}/\sim_L$ for the set of classes. It is a **right congruence**: if $u \sim_L v$ then $ua \sim_L va$ for every letter a.

The relation $\sim_L$ says that u and v have the same *future*: no continuation can tell them apart. If a machine is reading L and has seen u, the only thing it can usefully remember is $[u]_L$.



Theorem 3.41 (Myhill–Nerode)

Let $L \subseteq \mathbb{W}$ with $\varepsilon \notin L$. Then L is regular if and only if $\sim_L$ has finitely many equivalence classes. In that case the number of classes is the number of states of the minimal automaton for L.

Proof. ($\Rightarrow$) Let $L = \mathcal{L}(D)$ for a deterministic D with state set Q. If $\hat{\delta}(q_0, u) = \hat{\delta}(q_0, v)$ then for every w

$$uw \in L \iff \hat{\delta}(q_0, uw) \in F \iff \hat{\delta}(\hat{\delta}(q_0, u), w) \in F \iff \hat{\delta}(\hat{\delta}(q_0, v), w) \in F \iff vw \in L,$$

so $u \sim_L v$. Hence $u \mapsto \hat{\delta}(q_0, u)$ induces a well-defined surjection from a subset of Q onto $\mathbb{W}/\sim_L$, and there are at most $|Q|$ classes.

($\Leftarrow$) Suppose there are finitely many classes. Define

$$D_L = \big(\Sigma,\ \mathbb{W}/\sim_L,\ \delta_L,\ [\varepsilon]_L,\ F_L\big), \qquad \delta_L([u]_L, a) = [ua]_L, \qquad F_L = \{[u]_L \mid u \in L\}.$$

The transition function is well defined because $\sim_L$ is a right congruence, and F_L is a union of classes because $u \sim_L v$ with $w = \varepsilon$ gives $u \in L \iff v \in L$. Since $\varepsilon \notin L$ we have $[\varepsilon]_L \notin F_L$, so this is a legal automaton. An induction on $|w|$ gives $\hat{\delta}_L([\varepsilon]_L, w) = [w]_L$, whence

$$w \in \mathcal{L}(D_L) \iff [w]_L \in F_L \iff w \in L.$$

For the last claim: D_L is irreducible. Every state is accessible, since $[w]_L = \hat{\delta}_L([\varepsilon]_L, w)$. And if $[u]_L \neq [v]_L$ then some w has (say) $uw \in L$ and $vw \notin L$, and this w distinguishes the two states. So D_L is the minimal automaton, by Theorem 3.37. □

Remark. The hypothesis $\varepsilon \notin L$ is forced on us only by the convention $F \subseteq Q \setminus \{q_0\}$; without it the theorem holds verbatim for the usual definition of automaton in which the start state may accept.

Myhill–Nerode gives a second, often more convenient, method of proving non-regularity: exhibit infinitely many pairwise inequivalent words.

Example 3.42

Once more let $L = \{0^k 1^k \mid k > 0\}$. For $i \neq j$ the word 1^i distinguishes 0^i from 0^j, since $0^i 1^i \in L$ but $0^j 1^i \notin L$. So the words $0^0, 0^1, 0^2, \ldots$ lie in pairwise distinct classes, $\sim_L$ has infinitely many classes, and L is not regular.

Example 3.43

Let L be the set of nonempty words over $\{0,1\}$ containing at least one 0. If u contains a 0 then $uw \in L$ for every w; if u does not, then $uw \in L$ if and only if w contains a 0. So there are exactly two classes, and the minimal automaton has two states – exactly as we found by minimising.

Notice how much less work the second example was than the direct minimisation. This is typical: Myhill–Nerode lets us compute the minimal automaton by thinking about the language rather than by manipulating a machine.

§3.10 Decision Problems for Regular Languages

We can now settle all three of our decision problems for regular grammars. The word problem was dealt with in Corollary 2.16 (or, more sensibly, by running the automaton). The other two require a little work, and both are consequences of the pumping lemma and the finiteness of the machinery.

Lemma 3.44

Let L be a nonempty regular language with pumping number n. Then L contains a word of length less than n.

Proof. Take $w \in L$ of minimal length. If $|w| \geq n$ then $w = xyz$ with $|y| > 0$ and $xz = xy^0z \in L$, which is shorter, contradicting minimality. □

Corollary 3.45

The emptiness problem for regular grammars is solvable.

Proof. Given a regular grammar G, construct a deterministic automaton for $\mathcal{L}(G)$ and let n be its number of states, a pumping number by Theorem 3.20. By Lemma 3.44, $\mathcal{L}(G) = \varnothing$ if and only if $\mathcal{L}(G)$ contains no word of length less than n. There are finitely many such words, and the word problem is solvable, so we can check them all. □

Corollary 3.46

The finiteness problem for regular grammars – given G, is $\mathcal{L}(G)$ finite? – is solvable.

Proof. With n as above, we claim $\mathcal{L}(G)$ is infinite if and only if it contains a word w with $n \leq |w| < 2n$. If such a w exists then it can be pumped, and $\{xy^k z \mid k \in \mathbb{N}\}$ is an infinite subset of $\mathcal{L}(G)$. Conversely, if $\mathcal{L}(G)$ is infinite, take $w \in \mathcal{L}(G)$ of minimal length at least n; writing $w = xyz$ as in the pumping lemma we have $|y| \leq |xy| \leq n$, so $xz \in \mathcal{L}(G)$ has length $|w| - |y| \geq |w| - n$, and by minimality $|w| - n < n$. Again there are finitely many words to check. □

For the equivalence problem the plan is to build the minimal automaton for each language and check whether they are isomorphic, which by Theorem 3.37 settles the question. We must check that both steps can actually be carried out in finitely many steps.

Proposition 3.47

Given a deterministic automaton D and a state q, it is solvable whether q is accessible.

Proof. If $\hat{\delta}(q_0, w) = q$ for some w, then taking w of minimal length, the states visited must all be distinct (else we could excise a loop), so $|w| < |Q|$. There are finitely many such words, and we can compute $\hat{\delta}$ on each. □

Theorem 3.48 (The table-filling algorithm)

Given a deterministic automaton D and states q, q', it is solvable whether $q \sim q'$.

Proof. Form a table A indexed by unordered pairs of distinct states. Each entry is either blank or holds a word. The intended invariant is that $A_{q,q'}$ holds a word distinguishing q and q'.

Initialise. If exactly one of q, q' lies in F, set $A_{q,q'} = \varepsilon$.

Iterate. Repeatedly do the following sweep: for each blank entry $A_{q,q'}$ and each $a \in \Sigma$, if $\delta(q, a) \neq \delta(q', a)$ and the entry $A_{\delta(q,a),\delta(q',a)}$ holds a word w, then set $A_{q,q'} = aw$. Each sweep takes at most $|Q|^2|\Sigma|$ operations, and fills at least one blank entry or else nothing changes; since there are at most $|Q|^2$ entries, after at most $|Q|^2$ sweeps the table is stable.

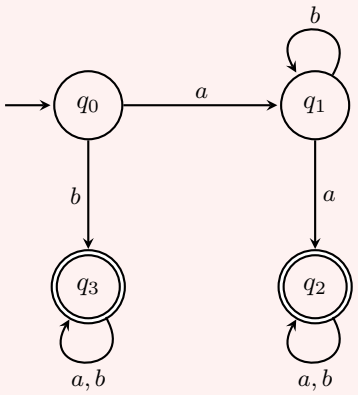
Correctness. If $A_{q,q'} = w$ then w genuinely distinguishes q and q': this is true at initialisation, and is preserved by the iteration, since $\hat{\delta}(q, aw) = \hat{\delta}(\delta(q, a), w)$.

Conversely suppose some distinguishable pair is left blank; among all such pairs choose q, q' with a distinguishing word w of minimal length. If $w = \varepsilon$ then the pair was filled at initialisation, a contradiction. So $w = av$. Then v distinguishes $\delta(q, a)$ from $\delta(q', a)$, and v is shorter than w, so by minimality that pair is not blank in the final table. But then the sweep would have filled $A_{q,q'}$, contradicting stability.

So on termination, $A_{q,q'}$ is blank precisely when $q \sim q'$. □

Example 3.49

Consider the automaton



After initialisation, the pairs involving exactly one of the accept states q_2, q_3 are marked with ε:

	q_0	q_1	q_2	q_3
q_0			ε	ε
q_1			ε	ε
q_2				
q_3				

In the first sweep, $\delta(q_0, a) = q_1$ and $\delta(q_1, a) = q_2$, and the entry A_{q_1,q_2} is marked, so we mark A_{q_0,q_1} with a:

	q_0	q_1	q_2	q_3
q_0		a	ε	ε
q_1			ε	ε
q_2				
q_3				

The only remaining blank is A_{q_2,q_3}, and it stays blank: both q_2 and q_3 are absorbing accept states, so $\delta(q_2, a) = q_2$, $\delta(q_3, a) = q_3$ and the entry A_{q_2,q_3} can only ever be filled from itself. So $q_2 \sim q_3$, and the minimal automaton has three states $[q_0], [q_1], [q_2]$. (Directly: $\mathcal{L}(D) = \mathcal{L}(b(a+b)^\star + ab^\star a(a+b)^\star)$, and one can check by hand that the three classes are 'nothing decided yet', 'an a has been seen', and 'accepted forever'.)

Corollary 3.50

The equivalence problem for regular grammars is solvable.

Proof. Given regular grammars G, G', construct deterministic automata for them; delete inaccessible states, which we can identify; and quotient by $\sim$, which we can compute by Theorem 3.48. This yields irreducible automata I, I'. Now $\mathcal{L}(G) = \mathcal{L}(G')$ if and only if $I \cong I'$ by Theorems 3.36 and 3.37, and since I and I' are finite there are only finitely many candidate bijections to test. □

For regular languages, then, everything is as good as it could be: all the closure properties hold, all three decision problems are solvable, and there is a canonical smallest machine. As we ascend the hierarchy, each of these will fail in turn.

§4 Context-Free Languages

We move one step up. A context-free rule replaces a single variable A by an arbitrary nonempty string, with no restriction on the shape of the right-hand side. We already know from Example 3.21 that this buys us something, since $\{0^k 1^k \mid k > 0\}$ is generated by the context-free grammar

$$S \to 0S1, \qquad S \to 01$$

but is not regular. The essential new feature is that a context-free derivation is no longer linear. In a regular grammar every intermediate string had the form wA, so a derivation was a list; here several variables can be present at once, and the natural object recording a derivation is a tree.

§4.1 Trees

We set up trees in the way that is most convenient for the arguments below, namely as sets of finite sequences of naturals: a node is identified with the route taken to reach it from the root.

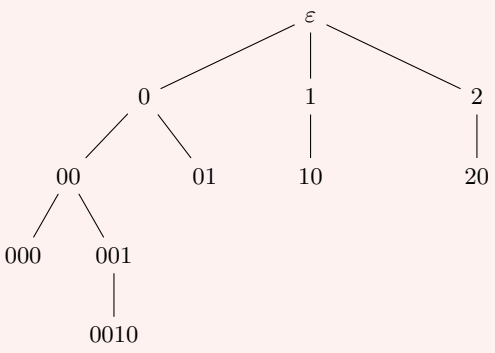
Definition 4.1 (Tree)

A set $T \subseteq \mathbb{N}^\star$ is a **(finitely branching) tree** if it is closed under initial segments, and for every $t \in T$ there is a **branching number** $n \in \mathbb{N}$ such that $tk \in T$ if and only if $k < n$.

The empty sequence is the **root**, and lies in every nonempty tree. A node with branching number 0 is a **leaf**. The **level** of t is $|t|$, and if T is finite the maximum level is the **height** of T. The **branch** to t is the sequence $t\restriction_0, t\restriction_1, \ldots, t\restriction_{|t|} = t$ of its initial segments.

Example 4.2

The set $T = \{\varepsilon, 0, 1, 2, 00, 01, 10, 20, 000, 001, 0010\}$ is a tree, drawn below with the root at the top.



Its height is 4, its leaves are $01, 10, 20, 000, 0010$, and the branch to 0010 is $\varepsilon, 0, 00, 001, 0010$.

Definition 4.3 (Subtree)

For $t \in T$, the **subtree at** t is $T_t = \{s \mid ts \in T\}$, which is again a tree.

Definition 4.4 (Left-to-right order)

Define $t < s$ for $t \neq s$ in T if, letting k_0 be least with $t(k_0) \neq s(k_0)$ (assuming such k_0 exists), we have $t(k_0) < s(k_0)$.

This is a partial order: it does not compare two nodes lying on the same branch, such as 0 and 00. It does, however, totally order the nodes at any fixed level, and in particular it totally orders the leaves. Reading the leaves in this order is what turns a tree into a string.

§4.2 Parse Trees

Definition 4.5 (Parse tree)

Let $G = (\Sigma, V, P, S)$ be a context-free grammar. A pair $\mathbb{T} = (T, \ell)$ is a **G-parse tree** if T is a finite tree and $\ell \colon T \to \Omega$ is a **labelling** such that

(i) $\ell(\varepsilon) \in V$, and we say that $\mathbb{T}$ **starts with** $\ell(\varepsilon)$;

(ii) if $\ell(t) \in \Sigma$ then t is a leaf;

(iii) if $\ell(t) = A \in V$ and t has branching number $n+1$, then $A \to \ell(t0)\ell(t1)\cdots\ell(tn)$ is a rule of P.

If $t_0 < t_1 < \cdots < t_m$ are the leaves of T in left-to-right order, the **string parsed by** $\mathbb{T}$ is

$$\sigma_{\mathbb{T}} = \ell(t_0)\ell(t_1)\cdots\ell(t_m).$$

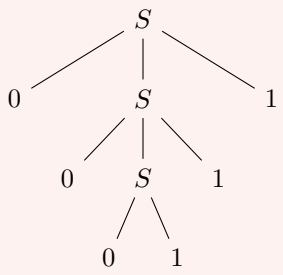
Note that a variable labelling a node of branching number 0 is permitted; such a leaf contributes a variable to $\sigma_{\mathbb{T}}$, and corresponds to a derivation that has not been finished. If $t \in T$, splitting the leaves into those to the left of t, those below t and those to the right of t gives

$$\sigma_{\mathbb{T}} = \alpha\, \sigma_{\mathbb{T}_t}\, \beta$$

for some strings α, β, where $\mathbb{T}_t = (T_t, \ell_t)$ with $\ell_t(s) = \ell(ts)$.

Example 4.6

Let G be the grammar $S \to 0S1 \mid 01$ over $\Sigma = \{0, 1\}$. The parse tree for 000111 is the following.



Its leaves in left-to-right order are the six letters $0, 0, 0, 1, 1, 1$, so $\sigma_{\mathbb{T}} = 000111$ as promised. The corresponding derivation is $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111$.

Proposition 4.7

Let G be context-free and $w \in \mathbb{W}$. Then $w \in \mathcal{L}(G)$ if and only if there is a G-parse tree $\mathbb{T}$ starting with S and with $\sigma_{\mathbb{T}} = w$.

Proof. Call a sequence $\mathbb{T}_0, \dots, \mathbb{T}_n$ of G-parse trees **derivative** if $\mathbb{T}_0 = (\{\varepsilon\}, \varepsilon \mapsto S)$, and each $\mathbb{T}_{i+1}$ is obtained from $\mathbb{T}_i$ by choosing a leaf t with $\ell_i(t) = A \in V$ and a rule $A \to x_0 \cdots x_n \in P$, and adjoining successors $t0, \dots, tn$ with $\ell_{i+1}(tk) = x_k$.

Derivative sequences correspond exactly to derivations from S: adjoining the successors of t replaces the occurrence of A contributed by t in $\sigma_{\mathbb{T}_i}$ by $x_0 \cdots x_n$, and leaves the rest of the string alone, which is precisely one application of $A \to x_0 \cdots x_n$. Hence $\sigma_{\mathbb{T}_0} \Rightarrow_G \sigma_{\mathbb{T}_1} \Rightarrow_G \cdots \Rightarrow_G \sigma_{\mathbb{T}_n}$, and conversely each derivation determines such a sequence by recording where each rule was applied.

So if there is a derivative sequence ending at $\mathbb{T}$, then $S \Rightarrow_G^\star \sigma_{\mathbb{T}}$. It remains to show every parse tree $\mathbb{T}$ starting with S arises this way. Set $\mathbb{T}_0$ to be the one-node tree. Given $\mathbb{T}_i$ with $T_i \subsetneq T$, pick $t \in T \setminus T_i$ of minimal length; then its predecessor $t\restriction_{|t|-1}$ is a leaf of T_i but not of T, and is labelled by a variable, so we may adjoin all of its T-successors to form T_{i+1}. Since T is finite this terminates with $T_n = T$, giving a derivative sequence. □

The following surgery on parse trees will be needed for the pumping lemma.

Definition 4.8 (Grafting)

Let $\mathbb{T} = (T, \ell)$ be a parse tree, $t \in T$ with $\ell(t) = A$, and $\mathbb{T}' = (T', \ell')$ a parse tree starting with A. Then $\operatorname{graft}(\mathbb{T}, t, \mathbb{T}') = (S, \ell^\star)$ where

$$S = \{s \in T \mid t \not\sqsubseteq s\} \cup \{tu \mid u \in T'\}, \qquad \ell^\star(s) = \begin{cases} \ell(s) & t \not\sqsubseteq s, \\ \ell'(u) & s = tu,\ u \in T'. \end{cases}$$

That is, we cut off the subtree at t and glue $\mathbb{T}'$ in its place; the result is again a parse tree, because the only condition involving t is that the rule applied at t matches its successors, and this is inherited from $\mathbb{T}'$. If $\sigma_{\mathbb{T}} = \alpha\sigma_{\mathbb{T}_t}\beta$ then

$$\sigma_{\operatorname{graft}(\mathbb{T},t,\mathbb{T}')} = \alpha\, \sigma_{\mathbb{T}'}\, \beta.$$

§4.3 Ambiguity

A word may have several parse trees, and this matters a great deal in practice, since the parse tree of a program is what determines its meaning.

Definition 4.9

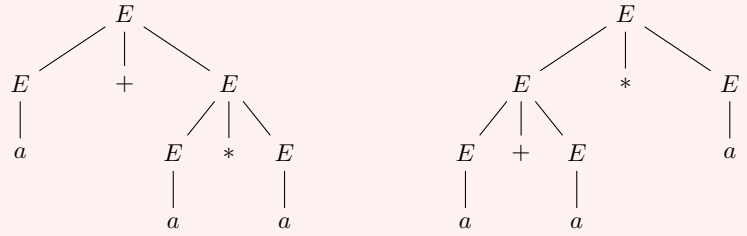
A context-free grammar G is **ambiguous** if some $w \in \mathcal{L}(G)$ has two distinct parse trees starting with S.

Example 4.10

Let $\Sigma = \{a, +, *, (,)\}$, $V = \{E\}$ and

$$E \to E + E \mid E * E \mid (E) \mid a.$$

The word $a + a * a$ has the two parse trees



both parsing $a+a*a$. The left tree groups the expression as $a+(a*a)$ and the right as $(a+a)*a$; if the letters were numbers these would evaluate differently. So G is ambiguous. One can find an unambiguous grammar for the same language by introducing variables for 'term' and 'factor', which is what forces the usual precedence rules; but there are context-free languages for which *every* grammar is ambiguous, and these are called **inherently ambiguous**.

§4.4 Chomsky Normal Form

Parse trees for general context-free grammars can have arbitrarily large branching, which makes counting arguments awkward. It is therefore very convenient that we may always assume the branching is at most 2.

Definition 4.11 (Chomsky normal form)

A grammar is in **Chomsky normal form** if every rule is of the form $A \to BC$ with $B, C \in V$ (a **binary** rule) or $A \to a$ with $a \in \Sigma$ (a **unary** rule).

Every grammar in Chomsky normal form is context-free. In such a grammar, parse trees are binary and derivations have a fixed length.

Lemma 4.12

Let G be in Chomsky normal form and $w \in \mathcal{L}(G)$ with $|w| = n$. Then every G-derivation of w from S has length exactly $2n - 1$.

Proof. A binary rule increases the length by one and the variable count by one; a unary rule leaves the length alone and decreases the variable count by one. We begin with the string S, of length 1 with one variable, and end with w, of length n with no variables. If p binary and q unary rules are used, then $1 + p = n$ and $1 + p - q = 0$, so $p = n - 1$ and $q = n$, giving $2n - 1$ steps. □

Theorem 4.13 (Chomsky)

Every context-free grammar is equivalent to one in Chomsky normal form, and such a grammar can be computed from the original.

Proof. There are three kinds of rule obstructing Chomsky normal form:

(a) rules $A \to \alpha$ with $|\alpha| \geq 2$ where α contains a letter;

(b) rules $A \to B$ with $B \in V$, called **unit rules**;

(c) rules $A \to \alpha$ with $|\alpha| > 2$ where α contains only variables.

We remove them in turn.

(a) For each $a \in \Sigma$ introduce a fresh variable X_a and the rule $X_a \to a$, and replace every offending rule $A \to \alpha$ by $A \to X(\alpha)$, where X replaces each letter a by X_a. This is exactly the construction of Lemma 2.19 applied to the right-hand sides, and the same argument shows the language is unchanged. Afterwards, the only rules whose right-hand side contains a letter are the unary rules.

(b) Call G **unit closed** if whenever $A \to B$ and $B \to \alpha$ lie in P, so does $A \to \alpha$. Every grammar can be made unit closed by repeatedly adding such rules; at most $|V| \cdot |P|$ new rules can be added, so this terminates, and it clearly does not change the language.

Now suppose G is unit closed, and let G' be G with all unit rules deleted. Certainly $\mathcal{L}(G') \subseteq \mathcal{L}(G)$. Conversely let $w \in \mathcal{L}(G)$ and take a G-derivation of w of minimal length. Suppose it uses a unit rule, and consider the *last* such use:

$$S \Rightarrow_G^\star \alpha A \beta \Rightarrow_G \alpha B \beta \Rightarrow_G^\star w.$$

Since w contains no variables, the occurrence of B just created must eventually be rewritten; let

$$S \Rightarrow_G^\star \alpha A \beta \Rightarrow_G \alpha B \beta \Rightarrow_G^\star \gamma B \delta \Rightarrow_G \gamma \zeta \delta \Rightarrow_G^\star w$$

where $B \to \zeta$ is the first rule applied to that occurrence. The segment $\alpha B \beta \Rightarrow_G^\star \gamma B \delta$ does not touch that occurrence of B, so the same rules apply verbatim to $\alpha A \beta$, giving $\alpha A \beta \Rightarrow_G^\star \gamma A \delta$. By unit closure $A \to \zeta \in P$, so we may continue $\gamma A \delta \Rightarrow_G \gamma \zeta \delta$, and we have produced a strictly shorter derivation of w – contradicting minimality. Hence the minimal derivation uses no unit rule, and is a G'-derivation.

(c) Finally consider $A \to A_1 A_2 \cdots A_n$ with $n > 2$ and all $A_i \in V$. Introduce fresh variables $X_1, \ldots, X_{n-2}$ and replace this rule by

$$A \to A_1 X_1, \quad X_1 \to A_2 X_2, \quad \ldots, \quad X_{n-3} \to A_{n-2} X_{n-2}, \quad X_{n-2} \to A_{n-1} A_n.$$

The new variables occur in no other rule, so the only strings derivable using them are the intermediate stages of the replacement, and the language is unchanged.

Applying (a), then (b), then (c) leaves only binary and unary rules. □

§4.5 The Pumping Lemma for Context-Free Languages

The pumping lemma for regular languages came from a repeated state on a path. Its analogue here comes from a repeated *variable* on a branch of a parse tree, and the resulting pump has two pieces – one on either side of the repeated subtree.

Definition 4.14

A language L satisfies the **context-free pumping lemma with pumping number** n if every $w \in L$ with $|w| \geq n$ can be written $w = xuyvz$ with

$$|uyv| \leq n, \qquad |uv| > 0, \qquad xu^k yv^k z \in L \text{ for all } k \in \mathbb{N}.$$

We call the pair (u, v) the **pump**.

Remark. Note the differences from the regular version. The pump has two parts, of which one may be empty (only their total length is required to be positive). There is no constraint on where the pump sits in the word: x and z may be arbitrarily long. But the pump together with the material between its two halves is bounded.

If L satisfies the regular pumping lemma with number n then it satisfies the context-free one with the same n, taking $v = \varepsilon$; so by Example 3.23 there are uncountably many languages satisfying the context-free pumping lemma, and it cannot possibly characterise the countably many context-free languages.

Theorem 4.15

Every context-free language satisfies the context-free pumping lemma for some n.

Proof. Let L be context-free. By Theorem 4.13 take a Chomsky normal form grammar G with $\mathcal{L}(G) = L$, put $m = |V|$ and $n = 2^m$.

Claim: if $\mathbb{T}$ is a G-parse tree of height $h+1$ whose string is a word, then $|\sigma_{\mathbb{T}}| \leq 2^h$. Since every node has branching number 1 or 2, and since $\sigma_{\mathbb{T}}$ is a word, the leaves are exactly the nodes labelled by letters and each arises from a unary rule applied to its parent. Deleting the leaves therefore gives a binary tree of height h, whose leaves are the parents; there are at most 2^h of these, and each has exactly one child. So $|\sigma_{\mathbb{T}}| \leq 2^h$.

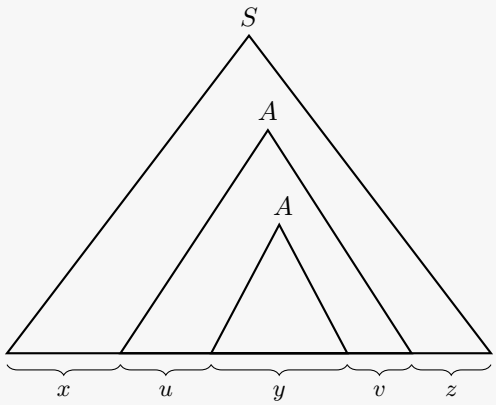
Now let $w \in L$ with $|w| \geq 2^m$ and let $\mathbb{T}$ be a parse tree for w. By the claim, the height of $\mathbb{T}$ is at least $m+1$. Let t be a node at maximal level, so the branch to t has $h+1 \geq m+2$ nodes; all but the last are labelled by variables, the last by a letter.

Let s be the node on that branch for which the subtree $\mathbb{T}_s$ has height exactly $m+1$. By the claim, $|\sigma_{\mathbb{T}_s}| \leq 2^m = n$. The portion of the branch from s to t has $m+2$ nodes, of which $m+1$ are labelled by variables; as there are only m variables, by pigeonhole two of them carry the same label. Say $t_0 \subsetneq t_1$ lie on this branch with $\ell(t_0) = \ell(t_1) = A$.

Decompose the string according to these nodes:

$$\sigma_{\mathbb{T}} = x' \, \sigma_{\mathbb{T}_s} \, z', \qquad \sigma_{\mathbb{T}_s} = x'' \, \sigma_{\mathbb{T}_{t_0}} \, z'', \qquad \sigma_{\mathbb{T}_{t_0}} = u \, \sigma_{\mathbb{T}_{t_1}} \, v, \qquad \sigma_{\mathbb{T}_{t_1}} = y,$$

and set $x = x'x''$ and $z = z''z'$, so that $w = xuyvz$. Since $\sigma_{\mathbb{T}_{t_0}} = uyv$ is a substring of $\sigma_{\mathbb{T}_s}$, we get $|uyv| \leq n$. And $t_0 \neq t_1$ with G in Chomsky normal form means the subtree at t_0 strictly contains that at t_1 together with at least one further leaf, so $|uv| > 0$.



Finally we pump. Both $\mathbb{T}_{t_0}$ and $\mathbb{T}_{t_1}$ are parse trees starting with A, so grafting is available. Set

$$\mathbb{T}_{(0)} = \mathbb{T}_{t_1}, \qquad \mathbb{T}_{(i+1)} = \mathrm{graft}(\mathbb{T}_{t_0}, t_1, \mathbb{T}_{(i)}),$$

so that by induction $\sigma_{\mathbb{T}_{(i)}} = u^i y v^i$. Then $\mathrm{graft}(\mathbb{T}, t_0, \mathbb{T}_{(k)})$ is a parse tree starting with S whose string is $xu^k yv^k z$, so $xu^k yv^k z \in L$ for every k. (For $k = 0$ this is grafting $\mathbb{T}_{t_1}$ in place of $\mathbb{T}_{t_0}$, which is the 'pump down'.) □

Example 4.16

The language $L = \{a^n b^n c^n \mid n > 0\}$ is not context-free.

Suppose it were, with pumping number N. Consider $a^N b^N c^N \in L$ and write it as $xuyvz$ with $|uyv| \leq N$ and $|uv| > 0$. Since uyv is a substring of length at most N, and the blocks of a's, b's and c's each have length N, the substring uyv cannot contain letters of all three kinds. Pumping up to $k = 2$ increases the number of occurrences of at least one letter (as $|uv| > 0$) but leaves the count of at least one other letter unchanged. The result is therefore not of the form $a^m b^m c^m$, contradiction.

Remark. The language $\{a^n b^n c^n\}$ *is* context-sensitive; a grammar for it may be built from rules such as $S \to abc \mid aAbc$, $Ab \to bA$, $Ac \to Bbcc$, $bB \to Bb$, $aB \to aa \mid aaA$, and one checks that this generates exactly L. So the inclusion of type 2 in type 1 is strict, as drawn in our picture of the hierarchy.

§4.6 Closure Properties

The context-free languages inherit closure under union and concatenation from Theorem 2.20, since context-free grammars are variable-based and the added rules $T \to SS'$, $T \to S$, $T \to S'$ are context-free. They are also closed under Kleene plus: given G, take a fresh T and put $T \to S \mid TS$.

The good news stops there.

Proposition 4.17

The context-free languages are not closed under intersection, complement or difference.

Proof. Let

$$L_0 = \{a^n b^n c^k \mid n, k > 0\}, \qquad L_1 = \{a^k b^n c^n \mid n, k > 0\}.$$

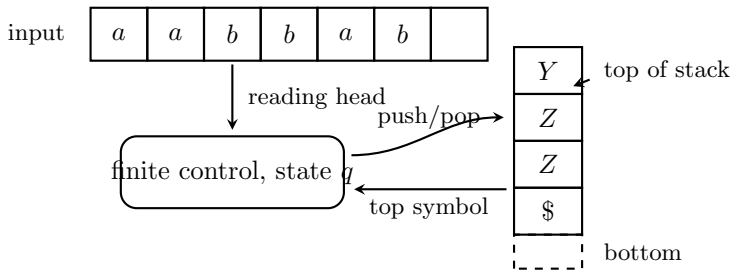
Both are context-free: L_0 is the concatenation of $\{a^n b^n\}$ and $\{c^k\}$, and L_1 is the concatenation of $\{a^k\}$ and $\{b^n c^n\}$, and each factor is context-free by the grammar of Example 4.6 or by a regular grammar. But $L_0 \cap L_1 = \{a^n b^n c^n \mid n > 0\}$, which is not context-free by Example 4.16.

Since the class is closed under union, closure under complement would give closure under intersection by De Morgan; so complement fails too, as does difference, since $L \cap M = L \setminus (L \setminus M)$. □

This has an important consequence for the machine model we are about to meet: *whatever* machines characterise the context-free languages, there can be no product construction for them. Two such machines cannot in general be run side by side on the same input. In hindsight the reason is easy to see, and it is that the machine will have an unbounded memory, so the pair of memories cannot be packaged into a single memory of the same kind.

§4.7 Pushdown Automata

We describe, without proofs, the machine model corresponding to context-free languages. A **pushdown automaton** is a finite automaton equipped with a **stack**: an unbounded sequence of symbols of which only the topmost is visible. Two operations are available, **push** (place a new symbol on top) and **pop** (remove the topmost symbol). There is no random access; a symbol placed on the stack can only be read after everything placed on top of it has been removed, so symbols come off in the reverse of the order in which they went on.



Formally, the transition function of such a machine takes as an extra input the symbol currently on top of the stack, and produces as an extra output an instruction to push a symbol, to pop, or to do nothing:

$$\delta \colon Q \times \Sigma \times \Gamma \longrightarrow \mathcal{P}\big(Q \times \Gamma^\star\big),$$

where Γ is a finite **stack alphabet**. The machine accepts if some run consumes the whole input and finishes in an accept state.

Theorem 4.18

A language is context-free if and only if it is accepted by a pushdown automaton.

It is worth seeing informally why the stack is exactly the right amount of extra power for $\{0^k 1^k\}$. Reading the 0s, the machine pushes one stack symbol per 0; reading the 1s,

it pops one per 1; and it accepts if the stack empties exactly as the input runs out. The stack cannot do this twice over – once emptied, the information is gone – which is the informal reason that $\{a^n b^n c^n\}$ is beyond its reach.

Note also that, unlike the case of finite automata, *nondeterministic* pushdown automata are strictly more powerful than deterministic ones: the deterministic pushdown automata accept a proper subclass of the context-free languages (for example the language of even-length palindromes is context-free but not deterministic context-free). So the pleasant state of affairs of Theorem 3.15 does not survive the move up the hierarchy either.

§4.8 Decision Problems for Context-Free Languages

Proposition 4.19

The word problem for context-free grammars is solvable.

Proof. Context-free grammars are noncontracting, so this is Corollary 2.16. Alternatively, and much more efficiently: convert to Chomsky normal form, whereupon by Lemma 4.12 every derivation of w has length exactly $2|w| - 1$, and check all of them. (The standard polynomial-time algorithm, due to Cocke, Younger and Kasami, computes for each substring of w the set of variables deriving it, by dynamic programming.) □

Proposition 4.20

The emptiness problem for context-free grammars is solvable.

Proof. Exactly as for regular languages. Let n be a context-free pumping number for $\mathcal{L}(G)$, computed as $2^{|V|}$ from a Chomsky normal form grammar. If $\mathcal{L}(G) \neq \varnothing$, take $w \in \mathcal{L}(G)$ of minimal length; if $|w| \geq n$ then $w = xuyvz$ can be pumped down to xyz, which is shorter and still in $\mathcal{L}(G)$, a contradiction. So $\mathcal{L}(G)$ is nonempty if and only if it contains a word of length less than n, and there are finitely many of those to test. □

Remark. The equivalence problem for context-free grammars, by contrast, is *unsolvable*; so is the question of whether a given context-free grammar is ambiguous, and whether the intersection of two context-free languages is empty. We shall not prove these, but the first taste of such a result is the unsolvability of the corresponding problems for type 0 grammars in Section 8.7.

We can summarise the situation so far in a table.

	union	concatenation	intersection	complement
regular	yes	yes	yes	yes
context-free	yes	yes	no	no

	word	emptiness	equivalence
regular	yes	yes	yes
context-free	yes	yes	no

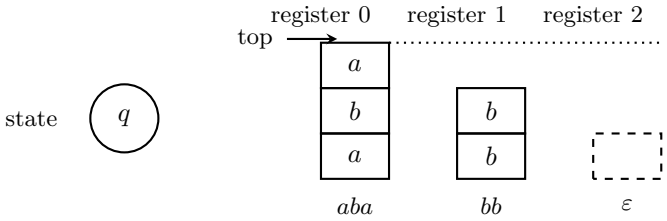
Everything above the line is settled; to go further we need to say what an algorithm is, and that is what we do next.

§5 Register Machines

We now give a precise definition of 'algorithm'. The model we use is the *register machine*: a finite automaton equipped not with one stack, but with finitely many. That extra freedom turns out to be all that is required to obtain full computational power.

§5.1 The Definition

A register machine has an alphabet Σ, a finite set Q of states, and finitely many **registers**, each holding a word $w \in \mathbb{W}$. The machine can inspect the last letter of a register, delete it, or append a new one – so a register behaves as a stack, with the right-hand end of the word as the top.



A snapshot of the whole machine is called a configuration.

Definition 5.1 (Configuration)

A **configuration** (or **snapshot**) of length $n+1$ is a tuple $(q, w_0, \dots, w_n) \in Q \times \mathbb{W}^{n+1}$.

One might be tempted to define the dynamics by a transition function $\delta\colon Q \times \mathbb{W}^{n+1} \to Q \times \mathbb{W}^{n+1}$, but this is far too permissive: there are uncountably many such functions, and they can do anything at all to the registers in one step. Instead we allow only four kinds of elementary instruction.

Definition 5.2 (Instruction)

Let Σ be an alphabet and Q a nonempty finite set of states. A **(Σ, Q)-instruction** is a tuple of one of the forms

$$(0,k,a,q) \in \mathbb{N}\times\mathbb{N}\times\Sigma\times Q, \qquad (1,k,a,q,q') \in \mathbb{N}\times\mathbb{N}\times\Sigma\times Q\times Q,$$
$$(2,k,q,q') \in \mathbb{N}\times\mathbb{N}\times Q\times Q, \qquad (3,k,q,q') \in \mathbb{N}\times\mathbb{N}\times Q\times Q.$$

We write these more legibly as

$$+(k,a,q), \qquad ?(k,a,q,q'), \qquad ?(k,\varepsilon,q,q'), \qquad -(k,q,q')$$

respectively, and we write $\mathrm{Instr}(\Sigma, Q)$ for the set of all of them.

Their intended meanings are as follows.

- $+(k,a,q)$: push the letter a onto register k, then move to state q.
- $?(k,a,q,q')$: if the top letter of register k is a, move to state q; otherwise move to q'.
- $?(k,\varepsilon,q,q')$: if register k is empty, move to state q; otherwise move to q'.

- $-(k, q, q')$: pop the top letter of register k. If the register was already empty, move to state q; otherwise move to q'.

Note that only $+$ and $-$ change the registers, and each changes exactly one register by exactly one letter.

Definition 5.3 (Register machine)

A tuple $M = (\Sigma, Q, P)$ is a **Σ-register machine** if Σ is an alphabet, Q is a finite set of states containing two distinguished states $q_S \neq q_H$ (the **start state** and the **halt state**), and

$$P \colon Q \to \mathrm{Instr}(\Sigma, Q)$$

is the **program**. Writing $Q = \{q_0, \ldots, q_n\}$, we present P as a finite list of **program lines** $q_i \mapsto P(q_i)$. Only finitely many register indices k occur in P; the largest is the **upper register index** of M.

Definition 5.4 (Computation)

Let M have upper register index m and let $C = (q, w_0, \ldots, w_m)$ be a configuration. We say M **transforms** C into C' if one of the following holds:

- $P(q) = +(k, a, q')$ and $C' = (q', w_0, \ldots, w_{k-1}, w_k a, w_{k+1}, \ldots, w_m)$;
- $P(q) = ?(k, a, q', q'')$ and either $w_k = wa$ for some w and $C' = (q', w_0, \ldots, w_m)$, or w_k does not end in a and $C' = (q'', w_0, \ldots, w_m)$;
- $P(q) = ?(k, \varepsilon, q', q'')$ and either $w_k = \varepsilon$ and $C' = (q', w_0, \ldots, w_m)$, or $w_k \neq \varepsilon$ and $C' = (q'', w_0, \ldots, w_m)$;
- $P(q) = -(k, q', q'')$ and either $w_k = \varepsilon$ and $C' = (q', w_0, \ldots, w_m)$, or $w_k = wa$ and $C' = (q'', w_0, \ldots, w_{k-1}, w, w_{k+1}, \ldots, w_m)$.

Given $\mathbf{w} \in \mathbb{W}^{m+1}$, the **computation sequence** of M on input $\mathbf{w}$ is defined by recursion:

$$C(0, M, \mathbf{w}) = (q_S, \mathbf{w}), \qquad C(k+1, M, \mathbf{w}) = \text{the transform of } C(k, M, \mathbf{w}).$$

If $\mathbf{w}$ is shorter than $m+1$ we pad it on the right with copies of ε; if it is longer we simply carry the extra registers along untouched.

Remark. Every configuration has a transform, so computation sequences as defined are always infinite. Halting is not the absence of a next step, but the arrival at a designated state.

Definition 5.5 (Halting)

The computation of M on input $\mathbf{w}$ **halts at time k** if k is least with $C(k, M, \mathbf{w}) = (q_H, \mathbf{v})$ for some $\mathbf{v}$; in this case $\mathbf{v}$ is the **output**. If no such k exists, the computation **does not halt**.

§5.2 Strong Equivalence

Definition 5.6

Register machines M and M' are **strongly equivalent** if for every input $\mathbf{w}$ and every k, the configurations $C(k, M, \mathbf{w})$ and $C(k, M', \mathbf{w})$ have the same register contents, and M halts on $\mathbf{w}$ at time k if and only if M' does.

So strongly equivalent machines do literally the same thing, step by step; only their state names may differ. Relabelling states shows that if $|Q| = |Q'|$ then every (Σ, Q, P) has a strongly equivalent (Σ, Q', P').

Proposition 5.7 (Padding lemma)

Every register machine is strongly equivalent to infinitely many distinct register machines.

Proof. Let $M = (\Sigma, Q, P)$ and add a fresh state $\hat{q}$ to Q, with $P(\hat{q})$ any instruction whatsoever. Since $\hat{q}$ occurs in no other program line, it never appears in any computation sequence, so the resulting machine is strongly equivalent to M. Repeating gives infinitely many. □

Proposition 5.8

Up to strong equivalence there are only countably many register machines over a fixed Σ.

Proof. Up to strong equivalence only $|Q|$ matters, so fix a set Q with $|Q| = n$ and bound the register index by k. Counting the four kinds of instruction,

$$|\mathrm{Instr}(\Sigma, Q)| = (k+1)n|\Sigma| + (k+1)n^2|\Sigma| + (k+1)n^2 + (k+1)n^2 =: N_{n,k},$$

which is finite, so there are $N_{n,k}^{n}$ programs. The class $M_{n,k}$ of such machines is therefore finite, and every register machine lies in some $M_{n,k}$; so $\bigcup_{n,k} M_{n,k}$ is a countable union of finite sets. □

This is the source of all the negative results in the course. There are only countably many algorithms, and uncountably many things one might want to compute.

§5.3 Operations and Questions

Rather than reason about programs line by line, we build up a library of things register machines can do, and thereafter argue at a higher level. There are two kinds of task.

Definition 5.9 (Operation)

An **operation** is a partial function $f\colon \mathbb{W}^{m+1} \rightharpoonup \mathbb{W}^{m+1}$. We write $f(\mathbf{w})\downarrow$ if $\mathbf{w} \in \operatorname{dom} f$ ('f **converges** at $\mathbf{w}$') and $f(\mathbf{w})\uparrow$ otherwise ('f **diverges**').

A register machine M **performs** f if for every $\mathbf{w}$ we have $f(\mathbf{w})\downarrow$ if and only if M halts on $\mathbf{w}$, and in that case the output of M is $f(\mathbf{w})$.

Example 5.10

The operation 'never halt' is the empty function. It is performed by any program which never mentions q_H on the right-hand side of a line, for instance

$$q_S \mapsto +(0, a, q_S), \qquad q_H \mapsto +(0, a, q_S).$$

The operation 'halt immediately, changing nothing' is the identity function on $\mathbb{W}^{m+1}$, performed by $q_S \mapsto ?(0, a, q_H, q_H)$: the test is performed, both branches lead to q_H, and nothing is written.

Definition 5.11 (Question)

A **question with $k+1$ answers** is a partition of $\mathbb{W}^{m+1}$ into sets $A_0, \dots, A_k$. A register machine **answers** the question if it has distinguished **answer states** $\bar{q}_0, \dots, \bar{q}_k$ such that on input $\mathbf{w}$ the computation reaches, after finitely many steps, the configuration $(\bar{q}_i, \mathbf{w})$, where i is the index with $\mathbf{w} \in A_i$.

So a question is answered without disturbing the registers, and the answer is recorded in the state.

Example 5.12

'Is register i empty?' is answered by the single line $q_S \mapsto ?(i, \varepsilon, \bar{q}_Y, \bar{q}_N)$, and 'does register i end in the letter a?' by $q_S \mapsto ?(i, a, \bar{q}_Y, \bar{q}_N)$.

The next two lemmas are what allow us to program at a high level: they say that machines can be run one after another, and that they can be selected between.

Lemma 5.13 (Concatenation)

If M performs F and M' performs F', then there is a register machine $\hat{M}$ performing $F' \circ F$.

Proof. Assume the state sets are disjoint. Let $P^\star$ be P with the line for q_H deleted and with every occurrence of q_H replaced by q'_S, and set $\hat{M} = (\Sigma, (Q \setminus \{q_H\}) \cup Q', P^\star \cup P')$, with start state q_S and halt state q'_H. Running $\hat{M}$ on $\mathbf{w}$ reproduces the computation of M exactly until the moment M would halt, at which point $\hat{M}$ is in state q'_S with registers $F(\mathbf{w})$, and thereafter reproduces the computation of M' on that input. So $\hat{M}$ halts precisely when both $F(\mathbf{w})\downarrow$ and $F'(F(\mathbf{w}))\downarrow$, with output $F'(F(\mathbf{w}))$. □

Lemma 5.14 (Case distinction)

Let Q be a question with answers $A_0, \dots, A_k$, answered by a machine M; and let F_i be operations performed by machines M_i. Then some register machine performs

$$G(\mathbf{w}) = F_i(\mathbf{w}) \quad \text{when } \mathbf{w} \in A_i.$$

Proof. Assume Q is disjoint from each Q_i and that the Q_i meet only in a common halt state q_H. Let $P_i^\star$ be P_i with every occurrence of $q_{S,i}$ replaced by the answer

state $\bar{q}_i$ of M, and delete the line for $q_{S,i}$. Put

$$Q^\star = Q \cup \bigcup_{i \le k} (Q_i \setminus \{q_{S,i}\}), \qquad P^\star = P \cup \bigcup_{i \le k} P_i^\star.$$

On input $\mathbf{w}$, the machine first runs M, which reaches $\bar{q}_i$ with the registers untouched, and from there runs M_i. □

§5.4 A Register Machine API

We may now assemble a library of operations, each built out of the previous ones by concatenation and case distinction. A register is **unused** if no program line mentions it, and **empty** if it contains ε; registers used only for intermediate work are called **scratch registers**. Since any given machine mentions only finitely many registers, an unused empty register is always available.

- *'Delete the last letter of register i'*: $q_S \mapsto -(i, q_H, q_H)$.
- *'Push a onto register i'*: $q_S \mapsto +(i, a, q_H)$.
- *'Empty register i'*: $q_S \mapsto -(i, q_H, q_S)$. This pops repeatedly, halting exactly when the register was found to be empty.
- *'Halt if register i is nonempty, otherwise diverge'*: answer 'is register i empty?', then apply case distinction with the operations 'never halt' and 'halt immediately'.
- *'Append the fixed word $w = a_0 a_1 \cdots a_\ell$ to register i'*: concatenate the $\ell + 1$ pushes.
- *'Replace register i by the fixed word w'*: empty it, then append w.
- *'What is the last letter of register i?'*: this question has $|\Sigma| + 1$ answers, one per letter and one for 'empty'. Ask 'does register i end in a_0?'; if yes go to $\bar{q}_0$, if no ask about a_1, and so on; if every answer is no, the register is empty and we go to $\bar{q}_\varepsilon$.
- *'Copy the last letter of register i onto register j'*: ask the previous question and push the corresponding letter in each case.
- *'Move the last letter of register i onto register j'*: copy it, then delete it from i.
- *'Move register i into register j in reverse order'*: repeat the previous operation until register i is empty.
- *'Move register i into register j in the correct order'*: move i into an unused empty register k in reverse order, then move k into j in reverse order. Reversing twice restores the order.
- *'Reverse register i'*: move it in reverse into an unused empty register, then move that back in the correct order – or equivalently, move it out and back once each.
- *'Move register i into both j and k in reverse order'*: copy the last letter into j, then into k, then delete it; repeat.
- *'Copy register i into register j'*: move i into j and a scratch register k simultaneously (in reverse), then move k back into i (in reverse). Composing with a reversal fixes the order if required.

- *'Is register i equal to the fixed word $w = a_0 \cdots a_\ell$?'*: run subroutines $S_\ell, S_{\ell-1}, \dots, S_0$, where S_r asks whether the last letter of register i is a_r; if not, go to a state q_N; if so, move that letter to a scratch register k and run S_{r-1}, or go to q_Y if $r = 0$. At q_N and q_Y, move the contents of k back into i (restoring it) and then answer $\bar{q}_N$ or $\bar{q}_Y$ respectively. Note that the scratch register must be restored, because a question is not allowed to alter the registers.

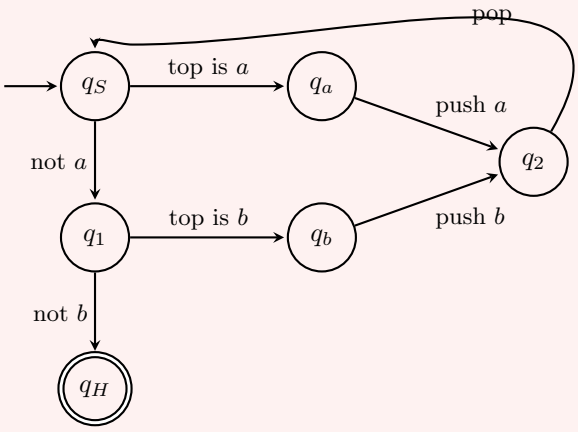
It is worth writing out one of these in full, to see that the formalism really does what we claim.

Example 5.15

Take $\Sigma = \{a, b\}$. The following five-line program moves the contents of register 1 into register 2 in reverse order, leaving register 1 empty.

$$q_S \mapsto ?(1, a, q_a, q_1), \quad q_1 \mapsto ?(1, b, q_b, q_H), \quad q_a \mapsto +(2, a, q_2), \quad q_b \mapsto +(2, b, q_2), \quad q_2 \mapsto -(1, q_S, q_S).$$

As a flow diagram:



Let us trace it with register 1 containing ab and register 2 empty. In q_S, the top letter is b, not a, so we move to q_1; there the top letter is b, so we move to q_b and push b onto register 2; in q_2 we pop, leaving register 1 holding a, and return to q_S. Now the top letter is a, so we go to q_a, push a onto register 2 (which now holds ba), and pop, leaving register 1 empty. Back at q_S: the top letter is not a (the register is empty), so we go to q_1; it is not b either, so we halt. Register 2 holds ba, the reverse of ab, as claimed.

From now on we shall specify register machines by describing what they do in English, in terms of the library above, exactly as one writes a program in terms of library functions rather than machine code. This is legitimate precisely because Lemmas 5.13 and 5.14 let us assemble the pieces.

§6 Computable Functions and Sets

§6.1 Computable Functions

Most computations need scratch space, and we do not want the final answer to depend on the litter left behind. So we declare that register 0 holds the output and that everything

else is scratch.

Definition 6.1

Let M be a register machine and $k \in \mathbb{N}$. Define $f_{M,k} \colon \mathbb{W}^k \rightharpoonup \mathbb{W}$ by

$$f_{M,k}(\mathbf{w}) = \begin{cases} v_0 & \text{if } M \text{ halts on } \mathbf{w} \text{ with output } \mathbf{v}, \\ \uparrow & \text{if } M \text{ does not halt on } \mathbf{w}. \end{cases}$$

A partial function $f \colon \mathbb{W}^k \rightharpoonup \mathbb{W}$ is **computable** if $f = f_{M,k}$ for some register machine M. For $k = 1$ we write $W_M = \operatorname{dom} f_{M,1}$.

If M and M' are strongly equivalent then $f_{M,k} = f_{M',k}$, though the converse fails badly: two machines may compute the same function in completely different ways.

Remark. By Proposition 5.8 there are only countably many computable functions, while there are uncountably many partial functions $\mathbb{W} \rightharpoonup \mathbb{W}$. So almost every function is not computable. On the other hand, by the padding lemma each computable function is computed by infinitely many machines. And by Lemmas 5.13 and 5.14, computable functions are closed under composition and case distinction.

Example 6.2

The identity $\mathbb{W} \to \mathbb{W}$ is computable (halt immediately). Each constant function $c(\mathbf{w}) = v$ is computable (replace register 0 by v). Each projection $\pi_i(\mathbf{w}) = w_i$ is computable (empty register 0, then move register i into register 0).

§6.2 Computable and Computably Enumerable Sets

To speak of computable *sets* we use characteristic functions. Since our values are words rather than truth values, we fix once and for all a letter $a \in \Sigma$ to stand for 'true', with ε standing for 'false'.

Definition 6.3

Let $X \subseteq \mathbb{W}^k$. A total $f \colon \mathbb{W}^k \to \mathbb{W}$ is **a characteristic function of** X if $f(\mathbf{w}) \neq \varepsilon$ exactly when $\mathbf{w} \in X$; it is **the characteristic function** χ_X if moreover $f(\mathbf{w}) = a$ for $\mathbf{w} \in X$. We call X **computable** if χ_X is computable.

A partial f is **a pseudocharacteristic function of** X if $\operatorname{dom} f = X$; it is **the pseudocharacteristic function** ψ_X if additionally $f(\mathbf{w}) = a$ for $\mathbf{w} \in X$. We call X **computably enumerable** if ψ_X is computable.

A computable set is one we can *decide*: given $\mathbf{w}$, the machine always halts and tells us the answer. A computably enumerable set is one we can only *semi-decide*: the machine halts when the answer is yes, and runs forever when the answer is no. Since a language is a set of words, both notions apply to languages.

Proposition 6.4

Let $X \subseteq \mathbb{W}^k$.

(i) X is computable if and only if its complement is computable.

(ii) X is computably enumerable if and only if $X = \operatorname{dom} f_{M,k}$ for some register machine M.

(iii) If X is computable then X is computably enumerable.

Proof. Write the proof for $k = 1$; the general case is identical. By case distinction, if g, h are computable then so is the function equal to $g(w)$ when $w \neq \varepsilon$ and $h(w)$ when $w = \varepsilon$, since 'is register 0 empty?' is answerable.

(i) Let f_1 be the computable function given in this way by $g(w) = \varepsilon$ and $h(w) = a$; that is, f_1 swaps a and ε. Then $\chi_{X^c} = f_1 \circ \chi_X$ and conversely.

(ii) If $X = \operatorname{dom} f$ with f computable, let f_2 be given by $g(w) = a$, $h(w) = a$, so f_2 is the total constant function a; then $\psi_X = f_2 \circ f$, which is computable with domain X. Conversely ψ_X is itself computable with domain X.

(iii) Let f_3 be given by $g(w) = a$ and $h(w) \uparrow$ (halt if register 0 is nonempty, else diverge). Then $\psi_X = f_3 \circ \chi_X$. □

The converse of (iii) is false, and proving so is the content of the halting problem below. First, a sanity check that our new notion is at least as strong as the old ones.

Theorem 6.5

Every regular language is computable.

Proof. Let $L = \mathcal{L}(D)$ with $D = (\Sigma, Q, \delta, q_0, F)$. We describe a register machine with input in register 0.

First, move register 0 into register 1 in reverse order. (This is necessary because a register machine sees the *last* letter of a word, whereas an automaton reads from the left; reversing makes the automaton's first letter the topmost.)

For each $q \in Q$ our machine has a block of states Q_q, meaning 'we are simulating D in state q'. Move into Q_{q_0}.

On entering Q_q: ask what the last letter of register 1 is. If register 1 is empty, move to a state $\bar{q}_{\text{acc}}$ if $q \in F$, and to $\bar{q}_{\text{rej}}$ otherwise. Otherwise, if the last letter is b, delete it and move into $Q_{\delta(q,b)}$, repeating.

Since each pass deletes a letter, the loop terminates after $|w|$ passes. At $\bar{q}_{\text{acc}}$, empty register 0, push a, and halt; at $\bar{q}_{\text{rej}}$, empty register 0 and halt. The resulting machine computes χ_L. □

§6.3 The Shortlex Ordering

Our functions take words as inputs, but recursion – both the definition of recursive functions in Section 7 and simple counting arguments – needs a notion of 'the next one'. We therefore transport the order type of $\mathbb{N}$ onto $\mathbb{W}$.

Definition 6.6 (Shortlex)

Fix a total order on Σ. The **shortlex order** on $\mathbb{W}$ is defined by $w < v$ if

(i) $|w| < |v|$; or

(ii) $|w| = |v|$, $w \neq v$, and at the least position where they differ, the letter of w precedes that of v.

So we sort by length first, and break ties lexicographically. This is a total order with least element ε.

Example 6.7

For $\Sigma = \{0, 1\}$ with $0 < 1$, the shortlex order begins

$$\varepsilon,\ 0,\ 1,\ 00,\ 01,\ 10,\ 11,\ 000,\ 001,\ 010,\ 011,\ 100,\ 101,\ 110,\ 111,\ 0000,\ \ldots$$

Identifying each word with its index (counting from 0), we have $\#\varepsilon = 0$, $\#10 = 5$, $\#01 = 4$, and since $\#010 = 9 = 5 + 4$ we may write $10 + 01 = 010$. Transporting the semiring structure of $\mathbb{N}$ in this way makes $\mathbb{W}$ into a commutative semiring; there are $|\Sigma|^k$ words of length k, so for $|\Sigma| = 2$ the index of 0^k is $2^k - 1$.

Theorem 6.8

$(\mathbb{W}, <)$ is order-isomorphic to $(\mathbb{N}, <)$.

Proof. For a fixed w the set $\{v \mid v < w\}$ is finite, since it consists of words of length at most $|w|$. So $\#(w) = |\{v \mid v < w\}|$ is a well-defined function $\mathbb{W} \to \mathbb{N}$, and it is clearly order-preserving. It is surjective, since the words listed in increasing order take every index in turn, and order-preserving maps between total orders are injective. □

Theorem 6.9

The set $\{(v, w) \mid v < w\}$ is computable, and the **successor function** $s \colon \mathbb{W} \to \mathbb{W}$ determined by $\#s(w) = \#w + 1$ is computable.

Proof. To compare v and w: copy them into scratch registers and repeatedly delete the last letter of each until one is empty; whichever empties first is shorter. If they empty simultaneously, copy them again in reverse order (so that we may read them from the left) and delete letters in step until they differ, then compare those letters in the fixed order on Σ; if no difference is found the words are equal. All of this uses only operations from Section 5.4.

For the successor: locate the last letter of w which is not the largest letter of Σ, replace it by its successor in Σ, and replace everything after it by copies of the least letter. If every letter of w is the largest letter (including the case $w = \varepsilon$), output the least letter repeated $|w| + 1$ times. □

§6.4 Merging and Splitting Words

We shall repeatedly need to package a pair of words into a single word. Recall the **Cantor zigzag function**

$$z(i,j) = \frac{(i+j)(i+j+1)}{2} + j,$$

a bijection $\mathbb{N} \times \mathbb{N} \to \mathbb{N}$; it enumerates the lattice points of the first quadrant along successive anti-diagonals.

Definition 6.10

For $v, w \in \mathbb{W}$, the **merge** $v * w$ is the unique word with $\#(v * w) = z(\#v, \#w)$. Conversely each w **splits** uniquely as $\#w = z(\#w_{(0)}, \#w_{(1)})$, and we call $w_{(0)}, w_{(1)}$ the components of w.

Since addition and multiplication of indices are computable (they can be built from the successor function by recursion, as we will see in Section 7), merging is computable; strictly speaking splitting is an *operation* rather than a computable function, since it has two outputs, but it is performed by a register machine which writes the components into two registers.

The point of merging is that a single existential quantifier over $\mathbb{W}$ can simulate two. We shall use this so often that it acquires a name: the **zigzag argument**.

§6.5 Coding of Programs

We now take the decisive step and treat programs as data. Let Σ be an alphabet and let Σ' be Σ together with ten new symbols

$$\mathbf{0} \quad \mathbf{1} \quad + \quad - \quad ? \quad (\quad) \quad , \quad \mapsto \quad \square.$$

For a mathematical object o we write $\mathrm{code}(o) \in \Sigma'^{\star}$ for its code, defined as follows.

- Naturals are written in binary using **0** and **1**, so $\mathrm{code}(19) = \mathbf{10011}$.
- For $Q = \{q_0, \dots, q_n\}$ we set $\mathrm{code}(q_k) = \mathrm{code}(k)$.
- Instructions are coded using the punctuation, for instance

  $$\mathrm{code}(+(k, a, \ell)) = + \,\mathbf{(}\, \mathrm{code}(k) \,\mathbf{,}\, a \,\mathbf{,}\, \mathrm{code}(\ell) \,\mathbf{)},$$

  and similarly for ? and $-$.
- A program line is coded by $\mathrm{code}(q \mapsto I) = \mathrm{code}(q) \mapsto \mathrm{code}(I)$, and a machine by listing its lines separated by **,**.
- A tuple of words is coded by $\mathrm{code}(\mathbf{w}) = \square w_0 \square w_1 \square \cdots \square w_k \square$, and a configuration by $\mathrm{code}(q, \mathbf{w}) = \mathrm{code}(q)\,\mathrm{code}(\mathbf{w})$.

Not every Σ'-word is a code, but it is a purely syntactic matter to check whether a given word is one, and this check can plainly be carried out by a register machine using the operations of Section 5.4.

Remark. We have enlarged the alphabet, which looks like cheating. It is not: if $\Sigma \subseteq \Sigma'$ then every Σ-machine is a Σ'-machine, and conversely one can encode each letter of Σ' as a fixed-length block of letters of Σ provided $|\Sigma| \geq 2$, translating Σ'-machines into Σ-machines. So computability over any alphabet with at least two letters is the same notion, and we may enlarge the alphabet whenever convenient.

§6.6 The Universal Register Machine

Lemma 6.11

The function

$$h(w,u,v) = \begin{cases} \text{code}\left(C(\#v, M, \mathbf{w})\right) & \text{if } w = \text{code}(M) \text{ and } u = \text{code}(\mathbf{w}) \text{ for some } M, \mathbf{w}, \\ \uparrow & \text{otherwise} \end{cases}$$

is computable.

Proof. First check syntactically that w codes a machine and u codes a tuple; if not, diverge. Then define h by recursion on v:

$$h(w,u,\varepsilon) = \text{code}(q_S)\, u, \qquad h(w,u,s(v)) = \text{code}(C'),$$

where C' is the transform, under the machine coded by w, of the configuration coded by $h(w,u,v)$. Computing C' from $\text{code}(C)$ and $\text{code}(M)$ is a finite syntactic manipulation: read off the current state from $\text{code}(C)$, look up the corresponding line of $\text{code}(M)$, and edit the appropriate register block accordingly. All of these are operations from Section 5.4, and the recursion itself is available because computable functions are closed under recursion (Theorem 7.8 below, whose proof does not depend on this lemma). □

Corollary 6.12

For each M and k the **truncated computation function**

$$t_{M,k}(\mathbf{w}, v) = \begin{cases} a & \text{if } M \text{ has halted on } \mathbf{w} \text{ by time } \#v, \\ \varepsilon & \text{otherwise} \end{cases}$$

is computable, and it is total.

Proof. Using h and recursion on v, examine the configurations at all times $\#u < \#v$; output a if any of them has state q_H, and ε otherwise. Every step of this terminates, so $t_{M,k}$ is total. □

The point of $t_{M,k}$ is that it converts a statement about halting – which we cannot decide – into a statement about halting *within a given time*, which we can.

Theorem 6.13 (The software principle)

The function

$$g(v,u) = \begin{cases} f_{M,k}(\mathbf{w}) & \text{if } v = \text{code}(M),\ u = \text{code}(\mathbf{w}) \text{ with } |\mathbf{w}| = k, \\ \uparrow & \text{otherwise} \end{cases}$$

is computable.

Proof. Check syntactically that v and u are codes of the right kind; if not, diverge. Let $f'(v,u,x)$ be the computable function returning a if the configuration $h(v,u,x)$ has state q_H, and ε otherwise; this is computable by Lemma 6.11.

Apply minimisation to f' (Theorem 7.8) to obtain a computable $\mu(v,u)$ returning the least x with $f'(v,u,x) = a$, and diverging if there is none. If M never halts on $\mathbf{w}$ then $\mu(v,u)\uparrow$, which is exactly what we want. If M does halt, then $\mu(v,u)$ converges to the halting time; compute $h(v,u,\mu(v,u))$, read off the block coding register 0, and write it into register 0. □

Definition 6.14

A register machine U with $f_{U,2} = g$ is called a **universal register machine**.

A universal register machine is a machine with a fixed, finite number of states and registers which can nonetheless simulate any register machine whatsoever, however many states and registers that machine has. This is the mathematical content of the statement that a computer can run any program: hardware is fixed, software is data.

It also lets us drop machines from the notation entirely. For $v \in \mathbb{W}$ write

$$f_{v,k}(\mathbf{w}) = f_{U,2}(v, \text{code}(\mathbf{w})),$$

so that $f_{v,k} = f_{M,k}$ whenever $v = \text{code}(M)$, and $f_{v,k}$ is the empty function when v is not a code. Write $W_v = \text{dom}\, f_{v,1}$; then $\{W_v \mid v \in \mathbb{W}\}$ is exactly the collection of computably enumerable subsets of $\mathbb{W}$.

§6.7 The s–m–n Theorem

The next result says that we may freeze one argument of a computable function of two arguments, and that a code for the resulting function can be computed from the frozen value. This operation is **currying**, after Haskell Curry.

Theorem 6.15 (s–m–n theorem, parameter theorem)

Let $g\colon \mathbb{W}^{k+1} \rightharpoonup \mathbb{W}$ be computable. Then there is a *total* computable $h\colon \mathbb{W} \to \mathbb{W}$ such that

$$f_{h(v),k}(\mathbf{w}) = g(\mathbf{w}, v) \qquad \text{for all } \mathbf{w}, v.$$

Proof. Fix v. The operation $\mathbf{w} \mapsto (\mathbf{w}, v)$ is performed by the register machine M_v which simply writes v into register k; and $\text{code}(M_v)$ is computable from v, since M_v has $|v|+1$ program lines each of which is written down mechanically from a letter of v. Let M be a machine with $f_{M,k+1} = g$.

By Lemma 5.13 the machine $M \circ M_v$ (first M_v, then M) performs the composite, and inspecting the proof of that lemma shows that $\text{code}(M \circ M_v)$ is computable from $\text{code}(M_v)$ and $\text{code}(M)$: one relabels states and concatenates lists of lines, both syntactic operations. So

$$h(v) = \text{code}(M \circ M_v)$$

is total and computable, and

$$f_{h(v),k}(\mathbf{w}) = f_{M \circ M_v,k}(\mathbf{w}) = g(\mathbf{w}, v). \qquad \square$$

Remark. The content is the computability of h. That *some* function h with $f_{h(v),k} = g(\cdot, v)$ exists is trivial, since $g(\cdot, v)$ is computable for each v and so has *some* code; the theorem says that a code can be found uniformly and effectively.

§6.8 The Halting Problem

Definition 6.16

Define

$$\mathbb{K}_0 = \{(w, v) \mid f_{w,1}(v)\!\downarrow\}, \qquad \mathbb{K} = \{w \mid f_{w,1}(w)\!\downarrow\}.$$

So $\mathbb{K}_0$ is the set of pairs (program, input) for which the program halts, and $\mathbb{K}$ is its diagonal.

Theorem 6.17

$\mathbb{K}_0$ and $\mathbb{K}$ are computably enumerable.

Proof. By Proposition 6.4(ii) it suffices to exhibit them as domains of computable functions. By the software principle, $f_{U,2}(w, v) = f_{w,1}(v)$, so $\operatorname{dom} f_{U,2} = \mathbb{K}_0$. The diagonal map $\Delta(w) = (w, w)$ is computable (copy register 0 into register 1), so $f_{U,2} \circ \Delta$ is computable with domain $\mathbb{K}$. $\square$

Theorem 6.18 (Unsolvability of the halting problem)

Neither $\mathbb{K}_0$ nor $\mathbb{K}$ is computable.

Proof. Suppose $\mathbb{K}_0$ were computable, so that $\chi_{\mathbb{K}_0}$ is a computable total function. Define

$$f(w) = \begin{cases} \uparrow & \text{if } \chi_{\mathbb{K}_0}(w, w) = a, \\ \varepsilon & \text{if } \chi_{\mathbb{K}_0}(w, w) = \varepsilon. \end{cases}$$

This is computable, by case distinction on the value of $\chi_{\mathbb{K}_0}(w, w)$ composed with the machines 'never halt' and 'halt immediately'. So $f = f_{d,1}$ for some $d \in \mathbb{W}$. But then

$$f(d)\!\downarrow \iff f_{d,1}(d)\!\downarrow \iff (d, d) \in \mathbb{K}_0 \iff \chi_{\mathbb{K}_0}(d, d) = a \iff f(d)\!\uparrow,$$

a contradiction. The argument for $\mathbb{K}$ is identical with $\chi_{\mathbb{K}}(w)$ in place of $\chi_{\mathbb{K}_0}(w, w)$. $\square$

Corollary 6.19

There is a computably enumerable set which is not computable, so the converse of Proposition 6.4(iii) fails.

This is the archetype of all unsolvability results, and its shape should be recognised: we assume a decision procedure exists, build from it a machine which is designed to disagree with itself, and then feed that machine its own code. The reader will recognise the diagonal argument of Cantor's theorem, and the self-reference of Gödel's incompleteness theorem, in the same shape.

§7 Recursive Functions

Register machines are one definition of computability. Church's, which came first, is completely different in spirit: instead of describing a machine, one specifies a small stock of obviously computable functions and a small stock of obviously computability-preserving constructions, and takes the closure. The remarkable fact – and the main theorem of this section – is that the two definitions agree exactly.

§7.1 Primitive Recursion and Minimisation

We work with partial functions $\mathbb{W}^k \rightharpoonup \mathbb{W}$ throughout, using the shortlex successor s of Theorem 6.9 in place of the successor on $\mathbb{N}$.

Definition 7.1 (Basic functions)

The **basic functions** are

$$\pi_{k,i} \colon \mathbb{W}^k \to \mathbb{W}, \quad \pi_{k,i}(\mathbf{w}) = w_i \qquad \text{(the \textbf{projections})},$$

$$c_{k,\varepsilon} \colon \mathbb{W}^k \to \mathbb{W}, \quad c_{k,\varepsilon}(\mathbf{w}) = \varepsilon \qquad \text{(the \textbf{constant} functions)},$$

$$s \colon \mathbb{W} \to \mathbb{W}, \quad \#s(w) = \#w + 1 \qquad \text{(the \textbf{successor})}.$$

Definition 7.2 (The three constructions)

Let $f, g, g_1, \dots, g_m$ be partial functions.

(i) (**Composition**) If $f \colon \mathbb{W}^m \rightharpoonup \mathbb{W}$ and $g_1, \dots, g_m \colon \mathbb{W}^k \rightharpoonup \mathbb{W}$, their composition is

$$h(\mathbf{w}) = f(g_1(\mathbf{w}), \dots, g_m(\mathbf{w})),$$

which is defined exactly when all the $g_i(\mathbf{w})$ are and f is defined at the resulting tuple.

(ii) (**Recursion**) If $f \colon \mathbb{W}^k \rightharpoonup \mathbb{W}$ and $g \colon \mathbb{W}^{k+2} \rightharpoonup \mathbb{W}$, the function $h \colon \mathbb{W}^{k+1} \rightharpoonup \mathbb{W}$ defined by

$$h(\mathbf{w}, \varepsilon) = f(\mathbf{w}), \qquad h(\mathbf{w}, s(v)) = g(\mathbf{w}, v, h(\mathbf{w}, v))$$

is said to be defined by recursion from f and g.

(iii) (**Minimisation**) If $f\colon \mathbb{W}^{k+1} \rightharpoonup \mathbb{W}$, the function $h\colon \mathbb{W}^k \rightharpoonup \mathbb{W}$ defined by

$$h(\mathbf{w}) = \begin{cases} v & \text{if } f(\mathbf{w}, u)\downarrow \text{ for all } u \leq v, \text{ and } v \text{ is least with } f(\mathbf{w}, v) = \varepsilon, \\ \uparrow & \text{if no such } v \text{ exists} \end{cases}$$

is said to be defined by minimisation from f.

A class $\mathcal{C}$ of partial functions is **closed** under one of these if the result of applying it to members of $\mathcal{C}$ again lies in $\mathcal{C}$.

Minimisation is the operation which introduces genuine partiality: it says 'search upwards through $\mathbb{W}$ for the first v making f vanish', and the search may never terminate. Note the careful clause requiring $f(\mathbf{w}, u)$ to converge for all $u \leq v$: the search must actually be carried out, and it cannot skip over a value at which f diverges.

Definition 7.3

A partial function is **recursive** (or **partial recursive**) if it lies in the smallest class containing the basic functions and closed under composition, recursion and minimisation. It is **primitive recursive** if it lies in the smallest class containing the basic functions and closed under composition and recursion.

The class of all partial functions is closed under all three constructions, so the definition makes sense.

Remark. Any class containing the basic functions and closed under composition contains all constant functions $c_{k,v}(\mathbf{w}) = v$: if $\#v = n$ then $c_{k,v} = s^n \circ c_{k,\varepsilon}$.

Every primitive recursive function is total, by induction: the basic functions are total, and composition and recursion of total functions are total. So minimisation is essential if we want to capture partial functions – and, as it happens, if we want to capture all total computable functions either, though we shall not prove that.

Example 7.4

Addition is primitive recursive. Interpreting the semiring structure on $\mathbb{W}$ transported from $\mathbb{N}$, we want h with $\#h(w, v) = \#w + \#v$. Take

$$h(w, \varepsilon) = \pi_{1,0}(w) = w, \qquad h(w, s(v)) = (s \circ \pi_{3,2})(w, v, h(w, v)) = s(h(w, v)).$$

Here $f = \pi_{1,0}$ is basic and $g = s \circ \pi_{3,2}$ is a composition of basic functions, so h is primitive recursive.

Multiplication is then obtained by recursion from addition, and exponentiation by recursion from multiplication, in the same way. With a little more work, so are the predecessor, truncated subtraction, comparison, the Cantor zigzag function and its inverses, and bounded sums, products and searches.

§7.2 Recursion Trees

The definition of 'recursive' is a closure, so proofs about recursive functions go by induction on the way a function is built. It is convenient to make the construction into a concrete object.

Attach to each construction a label, an **arity** and a **branching number**:

	label	arity	branching
projection	$B^{\pi}_{k,i}$	k	0
constant	B^{c}_{k}	k	0
successor	B^{s}	1	0
composition	$C_{n,k}$	k	$n+1$
recursion	R_k	$k+1$	2
minimisation	M_k	k	1

Definition 7.5 (Recursion tree)

A labelled tree (T, ℓ) is a **recursion tree** if the branching number of each node equals the branching number of its label, and

(i) if $\ell(t) = C_{n,k}$, the first successor of t has a label of arity n and the remaining n successors have labels of arity k;

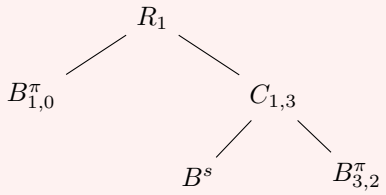
(ii) if $\ell(t) = R_k$, the first successor has arity k and the second has arity $k+2$;

(iii) if $\ell(t) = M_k$, the unique successor has arity $k+1$.

A recursion tree is **primitive** if no node is labelled M_k.

Example 7.6

The recursion tree for the addition function of Example 7.4 is



It has no minimisation nodes, so it is primitive, confirming that addition is primitive recursive.

Theorem 7.7

A partial function f is recursive if and only if $f = f_{T,\ell}$ for some recursion tree (T, ℓ), where $f_{T,\ell}$ is computed by reading the tree from the leaves upwards. It is primitive recursive if and only if it admits a primitive such tree.

Proof. Given a recursion tree, define $f_{T,\ell}$ by induction on the height of T: a leaf is labelled by a basic function, and an internal node applies the corresponding construction to the functions attached to its successors, the arity conditions guaranteeing that this makes sense.

Conversely, the class of functions of the form $f_{T,\ell}$ contains the basic functions (one-node trees) and is closed under the three constructions (join trees under a new root with the appropriate label), so it contains all recursive functions. The primitive case is identical. □

§7.3 Recursive Functions Are Computable

Theorem 7.8

The computable functions contain the basic functions and are closed under composition, recursion and minimisation. In particular every partial recursive function is computable.

Proof. The basic functions are computable: the projections and constants were dealt with in Section 6, and s in Theorem 6.9. Closure under composition is Lemma 5.13 (with a little bookkeeping to evaluate the g_i into separate scratch registers before applying f).

Recursion. Let f, g be computable and let h be defined by recursion from them; we describe a machine computing $h(\mathbf{w}, v)$. Let k and ℓ be unused empty registers; k will hold a counter and ℓ the current value.

(i) Compute $f(\mathbf{w})$ and store it in register ℓ. (If $f(\mathbf{w})\uparrow$ the machine diverges here, which is correct, since then $h(\mathbf{w}, v)\uparrow$ for every v.)

(ii) If $v = \varepsilon$, output the content of ℓ and halt.

(iii) Otherwise repeat the following. Let u be the content of ℓ and x the content of k. Compute $g(\mathbf{w}, x, u)$ and overwrite ℓ with the result. Then compare x with v: if $\#x = \#v - 1$, that is if $s(x) = v$, output ℓ and halt; otherwise apply s to register k and repeat.

After the jth pass through the loop, register ℓ holds $h(\mathbf{w}, s^j(\varepsilon))$, by induction; so the machine halts with $h(\mathbf{w}, v)$ if all the intermediate values converge, and diverges otherwise, as required.

Minimisation. Let f be computable and h obtained from it by minimisation. Let k be an unused empty register, holding ε initially. Repeat: compute $f(\mathbf{w}, u)$ where u is the content of k. If this diverges then the machine diverges, which is correct. If the result is ε, output u and halt. Otherwise apply s to register k and repeat. □

Remark. The proof gives more than the statement: the computable functions are *themselves* closed under recursion and minimisation, so we may use those constructions freely when building register machines. This is what we did in Lemma 6.11, Corollary 6.12 and Theorem 6.13, and there is no circularity, since the proof above uses none of those results.

§7.4 Computable Functions Are Recursive

The converse takes more work, and we sketch it; the details are routine but lengthy.

Theorem 7.9

Every computable function is partial recursive.

Proof sketch. Fix a register machine M with upper register index m, and code configurations as words as in Section 6; write $\gamma(k) = \mathrm{code}(C(k, M, \mathbf{w}))$ for the code of the configuration at time k on input $\mathbf{w}$.

Step 1: the transition is primitive recursive. The function taking $\mathrm{code}(C)$ to $\mathrm{code}(C')$, where M transforms C into C', is built from operations on words – extracting the state block, comparing it against each of the finitely many program lines of M, and appending or deleting a letter in one register block. Each of these is a bounded search or a case distinction over a fixed finite set, applied to substrings of a word whose length is bounded by that of the input, so each is primitive recursive. (Concretely, one codes a word by its shortlex index and uses the primitive recursive functions of Example 7.4 together with bounded minimisation, which preserves primitive recursiveness.)

Step 2: the whole computation is primitive recursive. The function $(\mathbf{w}, v) \mapsto \gamma(\#v)$ is obtained by recursion on v from Step 1, with the base case $\gamma(0) = \mathrm{code}(q_S, \mathbf{w})$, which is primitive recursive. Consequently

$$\theta(\mathbf{w}, v) = \begin{cases} \varepsilon & \text{if the state of } \gamma(\#v) \text{ is } q_H, \\ a & \text{otherwise} \end{cases}$$

is primitive recursive, since checking the state block is a comparison.

Step 3: one minimisation. Applying minimisation to θ yields $\mu(\mathbf{w})$, the least v such that M has reached q_H at time $\#v$, and $\mu(\mathbf{w})\uparrow$ exactly when M does not halt on $\mathbf{w}$. Finally

$$f_{M,k}(\mathbf{w}) = \rho\big(\gamma(\#\mu(\mathbf{w}))\big),$$

where ρ extracts the block coding register 0, and ρ is primitive recursive. So $f_{M,k}$ is partial recursive. □

Remark. The proof exhibits every computable function in the form

$$f(\mathbf{w}) = \rho\big(U(\mathbf{w}, \mu v.\, \theta(\mathbf{w}, v))\big)$$

with ρ, U, θ primitive recursive and a *single* application of minimisation. This is Kleene's normal form theorem, and it says that all the unboundedness of computation is concentrated in one unbounded search.

§7.5 The Church–Turing Thesis

There are other definitions of computability. **Turing machines** have a single two-way infinite tape and a read-write head which may move one cell at a time; **while programs** have variables and the control structures of an ordinary programming language, and halt simply when the instructions run out (so their computation sequences may be finite, unlike ours). Both give rise to notions of computable partial function.

Theorem 7.10

For a partial function $f\colon \mathbb{W}^k \rightharpoonup \mathbb{W}$, the following are equivalent:

(i) f is computable by a register machine;

(ii) f is partial recursive;

(iii) f is Turing computable;

(iv) f is while computable.

We have proved (i) $\iff$ (ii); the remaining equivalences are proved by simulation arguments of exactly the same character.

These four models look nothing like each other. One is a machine with stacks, one is a purely syntactic closure of a handful of functions, one is a machine with a tape, and one is a programming language. That they define the same class of functions is striking, and it is not an isolated coincidence: every model of computation which anyone has proposed and which is 'reasonable' – meaning, roughly, that each step is finitely describable and effects a bounded change – has turned out to define the same class.

Definition 7.11

The **Church–Turing thesis** is the assertion that any function computable by any effective procedure whatsoever is computable in the above sense.

This is not a mathematical statement and cannot be proved, since 'effective procedure' is not a mathematical notion; it is a claim that our formal definition has correctly captured an informal one. The evidence for it is the theorem above, together with ninety years of failure to find a counterexample. Its practical use is that, having accepted it, we may describe algorithms in English and conclude that the corresponding functions are computable, without ever writing down a register machine again. We shall do this freely from now on.

Remark. The thesis also lets us make our decision problems precise, and this is what we do in Section 8.7: a problem is *solvable* exactly when the associated set of codes is computable.

§8 Computably Enumerable Sets

§8.1 Sets with Quantifiers

There is a purely logical description of the computably enumerable sets, and it turns out to be the most useful way of recognising them in practice.

Definition 8.1

A set $X \subseteq \mathbb{W}^k$ is

- Σ_1 if there is a computable $Y \subseteq \mathbb{W}^{k+1}$ with

$$\mathbf{w} \in X \iff \exists y \, \big((\mathbf{w}, y) \in Y\big);$$

  we call $X = p(Y) = \{\mathbf{w} \mid \exists y \, (\mathbf{w}, y) \in Y\}$ the **projection** of Y;
- Π_1 if its complement is Σ_1;
- Δ_1 if it is both Σ_1 and Π_1.

Remark. The Σ is for the existential quantifier and the Π for the universal, following the analogy between sums and existentials, products and universals. The Δ is for *Durchschnitt*, 'intersection'. The clash with our use of Σ for the alphabet is unfortunate but standard; the subscript should keep them apart.

Proposition 8.2

Every computable set is Δ_1.

Proof. Computable sets are closed under complement, so it suffices to show every computable X is Σ_1. Take $Y = \{(\mathbf{w}, y) \mid \mathbf{w} \in X\}$, which is computable (ignore y), and note $p(Y) = X$: we have added a vacuous existential quantifier. □

Theorem 8.3

The computably enumerable sets are exactly the Σ_1 sets.

Proof. ($\Rightarrow$) Let X be computably enumerable, say $\psi_X = f_{M,k}$. Let

$$Y = \{(\mathbf{w}, y) \mid t_{M,k}(\mathbf{w}, y) = a\},$$

where $t_{M,k}$ is the truncated computation function of Corollary 6.12. Then $t_{M,k}$ is total computable and is a characteristic function of Y, so Y is computable. And

$$\mathbf{w} \in X \iff \psi_X(\mathbf{w})\downarrow \iff M \text{ halts on } \mathbf{w} \text{ at some time} \iff \exists y\,\big((\mathbf{w}, y) \in Y\big),$$

so $X = p(Y)$ is Σ_1.

($\Leftarrow$) Let $X = p(Y)$ with Y computable. Then Y^c is computable, so χ_{Y^c} is computable and total. Apply minimisation to χ_{Y^c}: the resulting $h(\mathbf{w})$ is the least y with $\chi_{Y^c}(\mathbf{w}, y) = \varepsilon$, i.e. the least y with $(\mathbf{w}, y) \in Y$, and diverges if there is none. So h is computable with $\operatorname{dom} h = p(Y) = X$, and X is computably enumerable by Proposition 6.4(ii). □

The utility of this is that quantifiers can be manipulated where machines cannot. Here is the basic move.

Example 8.4 (The zigzag argument)

Let $f\colon \mathbb{W}^2 \rightharpoonup \mathbb{W}$ be computable. Then $X = \{w \mid \exists v\ f(w, v)\downarrow\}$ is computably enumerable.

Note first that '$f(w, v)\downarrow$' is *not* a computable predicate, so we cannot simply quote the definition. Let $f = f_{M,2}$ and set

$$Z = \{(w, v_0, v_1) \mid t_{M,2}(w, v_0, v_1) = a\},$$

which is computable. Now merge the two existential quantifiers into one by merging the words:

$$Y = \{(w, u) \mid (w, u_{(0)}, u_{(1)}) \in Z\},$$

which is computable since splitting is performable. Then

$$\begin{aligned} w \in X &\iff \exists v_0\, \exists v_1\ (w, v_0, v_1) \in Z \\ &\iff \exists u\ (w, u_{(0)}, u_{(1)}) \in Z \\ &\iff \exists u\ (w, u) \in Y \iff w \in p(Y), \end{aligned}$$

so X is Σ_1.

The name comes from the picture: to search over pairs (v_0, v_1) we enumerate them along the anti-diagonals of $\mathbb{N}^2$, which lets us make progress on infinitely many computations at once by running each of them for a while. Any two consecutive existential quantifiers can be merged this way, and we use the technique without further comment.

Corollary 8.5

The computable sets are exactly the Δ_1 sets.

Proof. One direction is the proposition above. Conversely let X be Δ_1, so both X and X^c are computably enumerable, say $\psi_X = f_{M,k}$ and $\psi_{X^c} = f_{M',k}$. By Theorem 8.3,

$$\mathbf{w} \in X \iff \exists v\, t_{M,k}(\mathbf{w}, v) = a, \qquad \mathbf{w} \notin X \iff \exists v\, t_{M',k}(\mathbf{w}, v) = a.$$

Define

$$f(\mathbf{w}, v) = \begin{cases} t_{M,k}(\mathbf{w}, v_{(1)}) & \text{if } \#v_{(0)} \text{ is even,} \\ t_{M',k}(\mathbf{w}, v_{(1)}) & \text{if } \#v_{(0)} \text{ is odd,} \end{cases}$$

which is total and computable. Every $\mathbf{w}$ lies in exactly one of X, X^c, so at least one of the two searches succeeds; hence minimising f (with the roles of ε and a interchanged, so that we search for the first v with $f(\mathbf{w}, v) \neq \varepsilon$) always terminates, giving a total computable h. Output a if $\#h(\mathbf{w})_{(0)}$ is even and ε otherwise; this is χ_X. □

The idea is worth restating: if we can semi-decide both a set and its complement, we can decide it, by running both semi-decision procedures in parallel and waiting for one of them to answer. This is sometimes called *Post's theorem*.

Corollary 8.6

Σ_1 is not closed under complement.

Proof. $\mathbb{K}$ is Σ_1 but not computable, hence not Δ_1 by Corollary 8.5, hence not Π_1. So $\mathbb{W} \setminus \mathbb{K}$ is Π_1 but not Σ_1. □

§8.2 Closure Properties

Proposition 8.7

The computable sets are closed under union, intersection, complement, difference and concatenation.

Proof. For A, B computable, the function

$$\chi_{A\cap B}(\mathbf{w}) = \begin{cases} a & \chi_A(\mathbf{w}) = a \text{ and } \chi_B(\mathbf{w}) = a, \\ \varepsilon & \text{otherwise} \end{cases}$$

is computable: evaluate χ_A into a scratch register, then χ_B, then combine by case

distinction. Both terminate, since characteristic functions are total. Complement was Proposition 6.4(i), and union and difference follow.

For concatenation, let $A, B \subseteq \mathbb{W}$ and $w \in \mathbb{W}$. There are exactly $|w| + 1$ ways to write $w = vu$, and we can enumerate them; for each, evaluate $\chi_A(v)$ and $\chi_B(u)$, both of which terminate. Output a if some decomposition works. This is a bounded search and always terminates. □

Proposition 8.8

The computably enumerable sets are closed under union, intersection and concatenation, but not under complement or difference.

Proof. Failure of complement is Corollary 8.6; since $\mathbb{W} \setminus A = \mathbb{W} \setminus A$ is a difference, difference fails too.

Intersection: the function equal to a when $\psi_A(\mathbf{w})\downarrow$ and $\psi_B(\mathbf{w})\downarrow$, and undefined otherwise, is computed by running the two machines one after the other; if either diverges the composite diverges, which is exactly right.

Union: here we may *not* run them one after the other, since if ψ_A diverges we never reach ψ_B. Instead observe that

$$\mathbf{w} \in A \cup B \iff \exists v \left(t_{M,k}(\mathbf{w}, v) = a \text{ or } t_{M',k}(\mathbf{w}, v) = a\right),$$

with M, M' machines for ψ_A, ψ_B; the bracketed set is computable, so $A \cup B$ is Σ_1. Informally: run both machines in parallel, alternating steps, and halt as soon as either does.

Concatenation: let Z be the set of triples (w, v, u) such that v is an initial segment of w, say $w = vv'$, and both $\psi_A(v)$ and $\psi_B(v')$ have halted by time $\#u$. This is computable, using truncated computation functions. Setting $Y = \{(w, u) \mid (w, u_{(0)}, u_{(1)}) \in Z\}$ and zigzagging, $w \in AB$ if and only if $w \in p(Y)$. □

Proposition 8.9

$X \subseteq \mathbb{W}$ is computably enumerable if and only if $X = \operatorname{Im} f$ for some partial computable f.

Proof. If X is computably enumerable then

$$f(w) = \begin{cases} w & \psi_X(w)\downarrow, \\ \uparrow & \text{otherwise} \end{cases}$$

is computable with image X.

Conversely let $X = \operatorname{Im} f$ with $f = f_{c,1}$. Let Z be the set of triples (w, v, u) such that $t_{c,1}(v, u) = a$ and the output of the machine coded by c on input v, at the halting time, is w; this is computable, since given a bound on the running time we can simply run the machine. Then $w \in X$ if and only if $\exists v\, \exists u\ (w, v, u) \in Z$, and zigzagging gives a Σ_1 definition. □

Remark. A stronger statement is true: a nonempty X is computably enumerable if and only if $X = \operatorname{Im} f$ for a *total* computable f, so that $f(\varepsilon), f(s(\varepsilon)), f(s^2(\varepsilon)), \dots$ literally enumerates X, possibly with repetitions. This is what the name 'computably enumerable' refers to. The idea is to enumerate pairs (v, u) by zigzag and output the result of f on v if it has halted by time $\#u$, and some fixed element of X otherwise.

§8.3 Type 0 Languages

We can now identify the very top of the Chomsky hierarchy.

Theorem 8.10

Every type 0 language is computably enumerable.

Proof. Let $G = (\Sigma, V, P, S)$ and enlarge the alphabet to $\Sigma' = \Omega \cup \{\to\}$. Code a derivation $\sigma_0, \sigma_1, \dots, \sigma_n$ as the Σ'-word

$$\sigma_0 \to \sigma_1 \to \cdots \to \sigma_n,$$

and call a word of this shape a **derivation code** if consecutive σ_i really are related by a single application of a rule of P. Whether a given word is a derivation code is decidable: split it at the $\to$ symbols and, for each consecutive pair, search over the finitely many rules and the finitely many positions at which each could have fired. Let

$$Y = \{(w, v) \mid v \text{ is a derivation code with initial string } S \text{ and final string } w\},$$

which is therefore computable. Then $w \in \mathcal{L}(G)$ if and only if $\exists v\ (w, v) \in Y$, so $\mathcal{L}(G)$ is Σ_1. □

Theorem 8.11

Every computably enumerable $X \subseteq \mathbb{W}$ is a type 0 language.

Proof sketch. Use Theorem 7.10 and work with a Turing machine M computing ψ_X; arrange (by adding a clean-up phase) that the accepting computation on input w takes the tape from $q_S \square w \square$ to $q_H \square a \square$, with the head returned to the front.

A Turing machine *is* a rewrite system: each instruction 'in state q reading b, write c, move right, go to q'' is a rewrite rule on strings which contain a single state symbol marking the head position. So there is a rewrite system R with

$$q_S \square w \square \Rightarrow_R^\star q_H \square a \square \iff w \in X.$$

Now build a grammar which runs this backwards: start from S, produce $q_H \square a \square$, apply all the reversed instructions of M (which are again rewrite rules), and when the symbol q_S appears, delete everything except the word w sitting between the two $\square$s. Then $\mathcal{L}(G) = X$. Working backwards is essential, because a grammar derives *from* a start symbol; the details, which we omit, may be found in Salomaa's *Formal Languages*. □

Corollary 8.12

The type 0 languages are exactly the computably enumerable languages, that is, exactly the Σ_1 languages.

§8.4 Many-One Reducibility

Most of our unsolvability proofs so far have had the same shape: 'if this problem were solvable, so would be the halting problem'. We now name that shape.

Definition 8.13 (Reduction)

Let $A, B \subseteq \mathbb{W}$. A total computable $f\colon \mathbb{W} \to \mathbb{W}$ is a **reduction** from A to B if

$$w \in A \iff f(w) \in B \qquad \text{for all } w \in \mathbb{W},$$

that is, if $A = f^{-1}(B)$. We write $A \leq_m B$ if such an f exists, and say A is **many-one reducible** to B.

The subscript m records that f need not be injective. Intuitively $A \leq_m B$ says that A is no more complicated than B: any method for deciding membership in B yields one for A, by first applying f.

Proposition 8.14

The relation $\leq_m$ is reflexive and transitive, and respects complements: $A \leq_m B$ implies $\mathbb{W} \setminus A \leq_m \mathbb{W} \setminus B$. Moreover, if $A \leq_m B$ then

(i) if B is computable then so is A;

(ii) if B is computably enumerable then so is A.

Proof. Reflexivity uses the identity, transitivity composition. The same f witnesses the statement about complements, since $w \notin A \iff f(w) \notin B$. For (i) and (ii), if f is the reduction then $\chi_A = \chi_B \circ f$ and $\psi_A = \psi_B \circ f$, and composites of computable functions are computable. □

Contrapositively – and this is how the proposition is used – if $A \leq_m B$ and A is not computable then B is not computable. In particular:

$$\mathbb{K} \leq_m A \implies A \text{ is not computable}, \qquad \mathbb{W}\setminus\mathbb{K} \leq_m A \implies A \text{ is not computably enumerable}.$$

Note that $\leq_m$ is *not* antisymmetric, so it is a preorder rather than a partial order. As usual we may quotient: writing $A \equiv_m B$ when $A \leq_m B$ and $B \leq_m A$, the relation $\leq_m$ induces a genuine partial order on the equivalence classes, which are called the **many-one degrees**.

Proposition 8.15

Let A be computable and let $B \neq \varnothing, \mathbb{W}$. Then $A \leq_m B$.

Proof. Choose $v \in B$ and $u \notin B$. Since A is computable,

$$f(w) = \begin{cases} v & w \in A, \\ u & w \notin A \end{cases}$$

is total computable, and $w \in A \iff f(w) \in B$. □

So the computable sets (other than $\varnothing$ and $\mathbb{W}$, which are degenerate: nothing reduces to them but themselves) form the bottom degree.

§8.5 Degrees of Unsolvability

Definition 8.16

Let $\mathcal{C}$ be a class of subsets of $\mathbb{W}$. A set A is **$\mathcal{C}$-hard** if $B \leq_m A$ for every $B \in \mathcal{C}$, and **$\mathcal{C}$-complete** if in addition $A \in \mathcal{C}$.

A $\mathcal{C}$-complete set is a hardest member of $\mathcal{C}$: solving it solves everything in $\mathcal{C}$.

Corollary 8.17

Every computable A with $A \neq \varnothing, \mathbb{W}$ is Δ_1-complete.

Proof. The Δ_1 sets are the computable sets by Corollary 8.5, so this is Proposition 8.15. □

Theorem 8.18

$\mathbb{K}$ is Σ_1-complete.

Proof. We know $\mathbb{K} \in \Sigma_1$. Let X be any Σ_1 set, so X is computably enumerable, say $X = \operatorname{dom} f$ with f computable. Consider $g(w, u) = f(w)$, which is computable (ignore the second argument). By the s–m–n theorem there is a total computable h with

$$f_{h(w),1}(u) = g(w, u) = f(w).$$

We claim h reduces X to $\mathbb{K}$.

If $w \in X$ then $f(w)\downarrow$, so $f_{h(w),1}$ is the total constant function with value $f(w)$; in particular $f_{h(w),1}(h(w))\downarrow$, so $h(w) \in \mathbb{K}$.

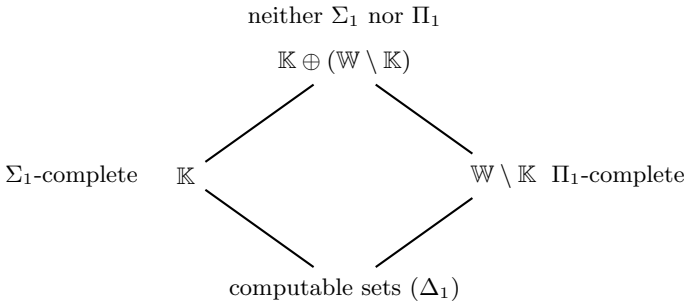
If $w \notin X$ then $f(w)\uparrow$, so $f_{h(w),1}(u)\uparrow$ for every u; in particular $f_{h(w),1}(h(w))\uparrow$, so $h(w) \notin \mathbb{K}$. □

Dually, $\mathbb{W} \setminus \mathbb{K}$ is Π_1-complete. And the two are genuinely different: $\mathbb{W} \setminus \mathbb{K} \not\leq_m \mathbb{K}$, since otherwise $\mathbb{W} \setminus \mathbb{K}$ would be computably enumerable by Proposition 8.14; and $\mathbb{K} \not\leq_m \mathbb{W} \setminus \mathbb{K}$ by taking complements. So there are at least three distinct degrees so far. There are more.

Definition 8.19

Suppose $\{0, 1\} \subseteq \Sigma$. The **join** of $A, B \subseteq \mathbb{W}$ is $A \oplus B = 0A \cup 1B$.

Then $A \leq_m A \oplus B$ via $w \mapsto 0w$, and likewise $B \leq_m A \oplus B$; and if $A \leq_m C$ and $B \leq_m C$ then $A \oplus B \leq_m C$, by case distinction on the first letter. So $\oplus$ computes least upper bounds in the partial order of degrees, and in particular $\mathbb{K} \oplus (\mathbb{W} \setminus \mathbb{K})$ is a set which is neither Σ_1 nor Π_1 but is above both.



Iterating the construction produces infinitely many degrees, and in fact the structure of the many-one degrees is extremely rich; this is the subject of computability theory proper, which we do not pursue.

§8.6 Rice's Theorem

The halting problem is unsolvable. Rice's theorem says, spectacularly, that *every* non-trivial question about the behaviour of a program is unsolvable.

Definition 8.20

Words v, u are **weakly equivalent**, written $v \sim u$, if $W_v = W_u$; that is, if the machines they code halt on exactly the same inputs. A set $I \subseteq \mathbb{W}$ is an **index set** if it is closed under weak equivalence.

An index set is thus a set of programs specified purely by the *function* they compute – more precisely, by its domain – rather than by any feature of the code. Index sets correspond exactly to properties of computably enumerable sets.

Example 8.21

$\varnothing$ and $\mathbb{W}$ are the trivial index sets. Others are

$$\textbf{Emp} = \{v \mid W_v = \varnothing\}, \quad \textbf{Fin} = \{v \mid W_v \text{ is finite}\}, \quad \textbf{Inf} = \{v \mid W_v \text{ is infinite}\}, \quad \textbf{Tot} = \{v \mid W_v = \mathbb{W}\}.$$

The emptiness problem for programs is precisely **Emp**.

The key construction is the following. Fix $w \in \mathbb{W}$ and consider

$$g(u, v) = \begin{cases} f_{w,1}(v) & \text{if } u \in \mathbb{K}, \\ \uparrow & \text{if } u \notin \mathbb{K}. \end{cases}$$

This looks like an illegal case distinction, since membership of $\mathbb{K}$ is not computable. But it *is* computable, because we do not need to decide the case: on input (u, v), run the machine coded by u on input u; if and when it halts, run the machine coded by w on input v and return the answer. If $u \notin \mathbb{K}$ the first phase never terminates, which produces exactly the divergence we want. This is a trick worth remembering: an uncomputable case distinction is harmless when the 'false' branch is divergence.

By the s–m–n theorem there is a total computable h with $f_{h(u),1}(v) = g(u,v)$, so

$$W_{h(u)} = \begin{cases} W_w & u \in \mathbb{K}, \\ \varnothing & u \notin \mathbb{K}. \end{cases}$$

Theorem 8.22 (Rice)

No non-trivial index set is computable.

Proof. Let I be an index set with $I \neq \varnothing, \mathbb{W}$. Fix $e \in \mathbb{W}$ with $W_e = \varnothing$ (for instance the code of a machine which never halts). Either $e \in I$ or not.

Case $e \in I$. Since $I \neq \mathbb{W}$, choose $w \notin I$; then $W_w \neq \varnothing$, since $W_w = \varnothing$ would give $w \sim e$ and hence $w \in I$. Build h as above from this w. If $u \in \mathbb{K}$ then $W_{h(u)} = W_w$, so $h(u) \sim w$ and $h(u) \notin I$; if $u \notin \mathbb{K}$ then $W_{h(u)} = \varnothing = W_e$, so $h(u) \sim e$ and $h(u) \in I$. Hence

$$u \in \mathbb{W} \setminus \mathbb{K} \iff h(u) \in I,$$

so $\mathbb{W} \setminus \mathbb{K} \leq_m I$ and I is not computable.

Case $e \notin I$. Since $I \neq \varnothing$, choose $w \in I$; again $W_w \neq \varnothing$. With the same h: if $u \in \mathbb{K}$ then $h(u) \sim w$ so $h(u) \in I$, and if $u \notin \mathbb{K}$ then $h(u) \sim e$ so $h(u) \notin I$. Hence $\mathbb{K} \leq_m I$, and again I is not computable. □

Remark. The proof gives more than the statement. If $e \in I$ then $\mathbb{W} \setminus \mathbb{K} \leq_m I$, so I is not even computably enumerable. If $e \notin I$ then $\mathbb{K} \leq_m I$, so I is not Π_1; but this leaves open whether I is computably enumerable.

Applying this: $e \in$ **Emp** and $e \in$ **Fin**, so neither **Emp** nor **Fin** is computably enumerable. On the other hand $e \notin$ **Inf** and $e \notin$ **Tot**, so we obtain only $\mathbb{K} \leq_m$ **Inf** and $\mathbb{K} \leq_m$ **Tot**.

Corollary 8.23

None of **Emp**, **Fin**, **Inf**, **Tot** is computable.

Theorem 8.24

Fin is neither Σ_1 nor Π_1.

Proof. By Remark , $\mathbb{W} \setminus \mathbb{K} \leq_m$ **Fin**, so **Fin** is not Σ_1. For the other half we show $\mathbb{K} \leq_m$ **Fin**. Consider

$$g(w,v) = \begin{cases} \uparrow & \text{if } t_{w,1}(w,v) = a, \\ \varepsilon & \text{otherwise,} \end{cases}$$

which is computable: compute the (total) truncated computation function and then either diverge or halt. By the s–m–n theorem take total computable h with $f_{h(w),1}(v) = g(w,v)$.

If $w \in \mathbb{K}$, let v_0 be the halting time of $f_{w,1}(w)$. Then $t_{w,1}(w,v) = a$ for all v with $\#v > \#v_0$, so $f_{h(w),1}(v)\uparrow$ for all such v; hence $W_{h(w)}$ is contained in the finite set

of v with $\#v \leq \#v_0$, and is finite. So $h(w) \in \mathbf{Fin}$.

If $w \notin \mathbb{K}$ then $t_{w,1}(w, v) = \varepsilon$ for every v, so $f_{h(w),1}$ is the total constant function ε and $W_{h(w)} = \mathbb{W}$, which is infinite. So $h(w) \notin \mathbf{Fin}$.

Hence h is a reduction from $\mathbb{K}$ to **Fin**, and **Fin** is not Π_1. □

Remark. By contrast, one can show that $\mathbf{Emp} \equiv_m \mathbb{W} \setminus \mathbb{K}$, so **Emp** is Π_1-complete, while **Tot** and **Inf** are, like **Fin**, in neither class. The hierarchy of Σ_n and Π_n sets obtained by allowing n alternating quantifiers – the *arithmetical hierarchy* – is where these sets properly live.

§8.7 Solvability of Decision Problems

We can now return to the decision problems of Section 4 with a precise definition in hand. Fix a coding of grammars such that every word w codes a grammar G_w and every grammar is G_w for some w; this is arranged by decreeing that words which are not syntactically valid code some fixed trivial grammar. Then

(i) the word problem is the set $\{(w, v) \mid w \in \mathcal{L}(G_v)\}$;

(ii) the emptiness problem is the set $\{w \mid \mathcal{L}(G_w) = \varnothing\}$;

(iii) the equivalence problem is the set $\{(w, v) \mid \mathcal{L}(G_w) = \mathcal{L}(G_v)\}$;

and each is **solvable** exactly when the corresponding set is computable.

Theorem 8.25

The word problem for type 0 grammars is unsolvable.

Proof. Let $W = \{(w, v) \mid w \in \mathcal{L}(G_v)\}$ and suppose it computable. Define

$$f(w) = \begin{cases} \uparrow & w \in \mathcal{L}(G_w), \\ a & w \notin \mathcal{L}(G_w). \end{cases}$$

Since W is computable this is a legitimate case distinction, so f is computable, and $\operatorname{dom} f$ is computably enumerable. By Theorem 8.11 there is a grammar G with $\mathcal{L}(G) = \operatorname{dom} f$, say $G = G_d$. But then

$$d \in \mathcal{L}(G_d) \iff d \in \operatorname{dom} f \iff f(d)\downarrow \iff d \notin \mathcal{L}(G_d),$$

a contradiction. □

Corollary 8.26

The emptiness problem for type 0 grammars is unsolvable.

Proof. By Theorems 8.10 and 8.11, $\{\mathcal{L}(G_w) \mid w \in \mathbb{W}\}$ is exactly the collection of computably enumerable sets, and translating between a grammar and a machine for the same language is effective in both directions. So the emptiness problem for grammars is many-one equivalent to **Emp**, which is not computable by Rice's theorem. □

Corollary 8.27

The equivalence problem for type 0 grammars is unsolvable.

Proof. Consider $\mathbf{Eq} = \{(w, v) \mid W_w = W_v\}$, and fix e with $W_e = \varnothing$. The function $g(w) = (w, e)$ is total computable, and

$$w \in \mathbf{Emp} \iff W_w = \varnothing = W_e \iff (w, e) \in \mathbf{Eq},$$

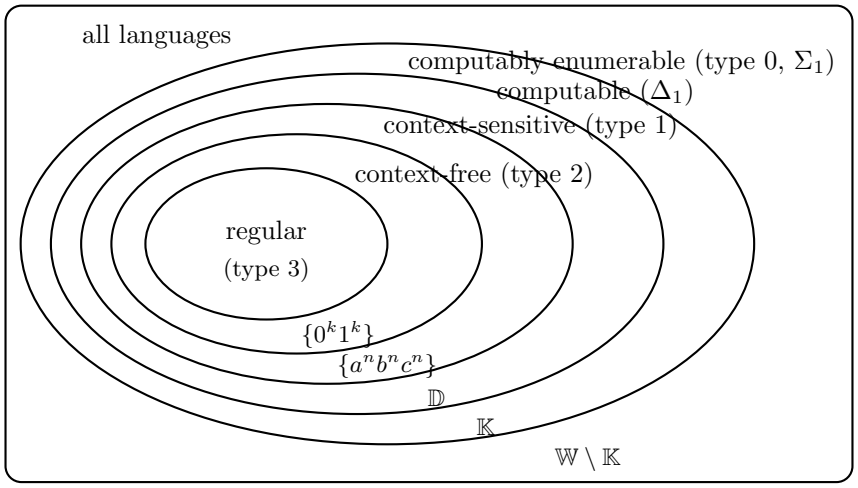
so $\mathbf{Emp} \leq_m \mathbf{Eq}$ and $\mathbf{Eq}$ is not computable. Translating to grammars as above gives the result. □

So all three of our problems, solvable for regular grammars and two of them for context-free grammars, become unsolvable at the top of the hierarchy.

§9 The Relationship Between the Language Classes

We close by drawing the whole picture together. We have met the following classes of languages over a fixed alphabet Σ, in increasing order:

regular $\subseteq$ context-free $\subseteq$ context-sensitive $\subseteq$ computable $\subseteq$ computably enumerable $\subseteq \mathcal{P}(\mathbb{W})$.



Every inclusion is strict, and each strictness has been (or can be) witnessed:

- $\{0^k1^k \mid k > 0\}$ is context-free but not regular, by the pumping lemma for regular languages (Example 3.21) or by Myhill–Nerode.
- $\{a^nb^nc^n \mid n > 0\}$ is context-sensitive but not context-free, by the pumping lemma for context-free languages (Example 4.16).
- There is a computable language $\mathbb{D}$ which is not context-sensitive. Indeed, the context-sensitive grammars can be effectively enumerated as $G_0, G_1, \ldots$, and the word problem for each is uniformly solvable by Corollary 2.16; so, listing the words of $\mathbb{W}$ in shortlex order as $w_0, w_1, \ldots$, the diagonal language

  $$\mathbb{D} = \{w_n \mid w_n \notin \mathcal{L}(G_n)\}$$

  is computable but differs from every $\mathcal{L}(G_n)$.

- $\mathbb{K}$ is computably enumerable but not computable (Theorem 6.18).
- $\mathbb{W} \setminus \mathbb{K}$ is a language which is not computably enumerable, hence not of the form $\mathcal{L}(G)$ for any grammar at all (Corollary 8.6).
- Finally, there are uncountably many languages and only countably many grammars, so almost every language lies outside the outermost ellipse.

Closure properties and decision problems degrade as we ascend:

	union	concat.	Kleene +	intersection	complement
regular	yes	yes	yes	yes	yes
context-free	yes	yes	yes	*no*	*no*
context-sensitive	yes	yes	yes	yes	yes
computable	yes	yes	yes	yes	yes
computably enumerable	yes	yes	yes	yes	*no*

	word problem	emptiness	equivalence
regular	solvable	solvable	solvable
context-free	solvable	solvable	*unsolvable*
context-sensitive	solvable	*unsolvable*	*unsolvable*
computably enumerable	*unsolvable*	*unsolvable*	*unsolvable*

The context-free row is anomalous in the first table: it is the only class in the list failing to be closed under intersection, and we saw in Section 4 that this is forced by the pushdown model, whose single stack cannot be duplicated. The last row of the second table is the content of Section 8.7.

There is a pleasing overall moral. Each class is characterised in two ways, by a grammar and by a machine:

class	grammars	machines
regular	regular grammars	finite automata
context-free	context-free grammars	pushdown automata
context-sensitive	noncontracting grammars	linear bounded automata
computably enumerable	arbitrary grammars	register (or Turing) machines

and the amount of memory available to the machine – none, a stack, a tape of length proportional to the input, an unbounded tape – is exactly what determines the class. That is the shape of the subject: a hierarchy of languages, a matching hierarchy of machines, and at the top, an unavoidable wall of undecidability.